

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

BÁO CÁO KẾT THÚC HỌC PHẦN
Học kỳ I/2025-2026
HỌC PHẦN: KIẾN TRÚC VÀ GIAO THỨC IOT

**THIẾT KẾ MÔ HÌNH GIÁM SÁT MÔI TRƯỜNG VÀ CUNG
CẤP THỨC ĂN TỰ ĐỘNG TRONG HỒ CÁ TẠI NHÀ**

Giáo viên hướng dẫn: Ths. Trịnh Quốc Thanh

Nhóm thực hiện: Nhóm 4

Mã lớp học: ITAP338264_01

TP. Hồ Chí Minh, tháng 10 năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



BÁO CÁO KẾT THÚC HỌC PHẦN
Học kỳ I/2025-2026
HỌC PHẦN: KIẾN TRÚC VÀ GIAO THỨC IOT

THIẾT KẾ MÔ HÌNH GIÁM SÁT MÔI TRƯỜNG VÀ CUNG CẤP THỨC ĂN TỰ ĐỘNG TRONG HỒ CÁ TẠI NHÀ

Giáo viên hướng dẫn: Ths. Trịnh Quốc Thanh

Nhóm thực hiện: Nhóm 4

Mã lớp học: ITAP338264_01

TP. Hồ Chí Minh, tháng 10 năm 2025

TP. HCM, ngày tháng năm 2024

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ MÔN
KIẾN TRÚC GIAO THỨC IOT
HỌC KỲ 1 NĂM HỌC 2025–2026

- Mã lớp môn học: ITAP338264_01
- Giảng viên hướng dẫn: ThS. Trịnh Quốc Thanh
- Tên đề tài: Thiết kế mô hình giám sát môi trường và cung cấp thức ăn tự động trong hồ cá tại nhà
- Danh sách nhóm:

STT	HỌ VÀ TÊN SINH VIÊN	Mã số sinh viên	Tỷ lệ % tham gia	Số điện thoại	Kí tên
01	Đoàn Minh Duy Bình	23139005		0971439154	
02	Hoàng Ngọc Dung	23139006		0971439154	
03	Trần Hữu Dương	23139009		0355097634	

Ghi chú:

- Tỷ lệ % = 100%
- Trưởng nhóm: Hoàng Ngọc Dung (SĐT: 0971439154)

Nhận xét của giáo viên

.....

.....

.....

.....

.....

.....

Ngày tháng năm 2025

Giáo viên chấm điểm

LỜI CAM ĐOAN

Chúng tôi xin cam đoan đề tài nghiên cứu là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những kết quả số liệu, thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ các nguồn khác nhau, có ghi rõ nguồn gốc. Tất cả các thông tin số liệu, bảng, hình ảnh và kết quả nghiên cứu được đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay chỉnh sửa nào có thể làm ảnh hưởng đến tính chính xác của kết quả.

Các nguồn dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và đã được ghi rõ nguồn gốc. Bất kỳ tài liệu tham khảo, nghiên cứu trước đây hoặc ý kiến cá nhân được sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Chúng tôi xin chịu hoàn toàn trách nhiệm, kỷ luật của bộ môn và nhà trường nếu như có bất cứ vấn đề nào.

LỜI CẢM ƠN

Lời đầu tiên, chúng tôi xin gửi lời cảm ơn chân thành đến Ths. Trịnh Quốc Thanh giảng viên bộ môn Kiến trúc và giao thức IoT lớp ITAP338264_01. Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài nghiên cứu này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng tôi những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

Hồ Chí Minh, tháng 7 năm 2024

MỤC LỤC

DANH MỤC HÌNH VẼ.....	1
DANH MỤC BẢNG BIỂU	2
DANH MỤC CÁC CHỮ VIẾT TẮT	3
MỞ ĐẦU.....	4
GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI	4
1. Giới thiệu.....	4
2. Mục tiêu của đề tài	5
3. Đối tượng, phạm vi nghiên cứu	5
4. Phương pháp nghiên cứu	6
CHƯƠNG 1 CƠ SỞ LÝ THUYẾT	8
1.1. Mô hình giám sát chất lượng nước và cung cấp thức ăn tự động.....	8
1.1.1. Mô hình giám sát chất lượng nước, độ đục và cho cá ăn	8
1.1.2. Nguyên lý đo độ đục của nước	9
1.1.3. Hình thức cung cấp thức ăn.....	10
1.2. Cơ sở dữ liệu	11
1.2.1. Firsebase	11
1.3. UI (User interface) và UX (User experience)	12
1.3.1. HTML (HyperText Markup Language)	12
1.3.2. CSS (Cascading Style Sheets)	12
1.3.3. JavaScript (JS)	13
1.3.4. Flask (micro web framework for Python)	14
1.4. Các chuẩn truyền thông	15
1.4.1. Các chuẩn truyền thông không dây	15
1.4.2. Các chuẩn truyền thông có dây	15
1.4.3. Giao thức IoT: MQTT	17
1.5. Thiết bị phần cứng.....	23
1.5.1. ESP8266 ESP-01	23
1.5.2. Relay SRD-5VDC-SL-C	24
1.5.3. SG90 Động Cơ Servo 180 Độ	25
1.5.4. Cảm biến nhiệt độ MKE-S15 DS18B20	26
1.5.5. Cảm biến đo độ đục MJKDZ Turbidity Sensor	28

1.5.5. Cảm biến siêu âm HCSR04	29
CHƯƠNG 2 NGHIÊN CỨU, THIẾT KẾ MÔ HÌNH.....	30
2.1. Quy trình các bước thực hiện	30
2.2. Mô hình tổng quan hệ thống.....	32
2.2. Sơ đồ khối hệ thống	34
2.3. Sơ đồ chi tiết khối chính.....	35
2.3.1. Khối nguồn (Bình).....	35
2.3.2. Khối cảm biến.....	37
2.3.3. Khối xử lý trung tâm	37
2.4. Quy trình các bước thực hiện	38
2.4.1. Xác định yêu cầu và linh kiện	38
2.4.2. Sơ đồ nguyên lý	39
2.4.3. Thiết kế cơ khí	39
2.4.4. Thiết kế nhúng	40
2.4.5. Thiết kế web	49
2.4.6. Hiệu chuẩn và kiểm thử (to be continue)	60
CHƯƠNG 3 THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ	61
3.1. Kết quả thiết kế phần cứng.....	61
3.1. Kết quả thiết kế phần mềm.....	62
3.2. Mô phỏng một số kịch bản.....	63
3.3. Nhận xét, đánh giá và kết quả	64
CHƯƠNG 4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	66
4.1. Kết luận	66
4.2. Hướng phát triển	66
TÀI LIỆU THAM KHẢO.....	69

DANH MỤC HÌNH VẼ

Hình 1: Tổng quan mô hình giám sát chất lượng nước và cho ăn tự động	9
Hình 2: Sơ đồ minh họa nguyên lý hoạt động của cảm biến độ đục	10
Hình 3: Cách kết nối của chuyên truyền SPI.....	16
Hình 4: Cách kết nối của chuẩn truyền UART.....	17
Hình 5: Cảm biến nhiệt độ MKE-S15 DS18B20	26
Hình 6: Cảm biến đo độ đục MJKDZ Turbidity Sensor	28
Hình 7: Mô hình chữ V.	30
Hình 8: Sơ đồ mô hình tổng quan sử dụng MQTT vào ESP8266 kết hợp STM32	32
Hình 9: Kiến trúc tổng quan của hệ thống iot 3 lớp cho việc giám sát và cung cấp thức ăn tự động cho cá.....	34
Hình 10: Sơ đồ hệ thống.....	34
Hình 11: Mô hình được cấp nguồn từ bộ pin Li-ion 3.7 V (2 cell, nối tiếp qua module sạc – chuyển áp), cung cấp điện cho vi điều khiển (ESP32/STM32) và các cảm biến thông qua mạch ổn áp 5 V – 3.3 V.....	36
Hình 12: Sơ đồ chân các cảm biến trong mô hình.....	37
Hình 13: Giá đỡ phần giữ thức ăn và servo.....	40
Hình 14: Cơ khí cho ăn đã gắn servo và que chặn	40
Hình 15: Sơ đồ Use Case các tính năng của mô hình	41
Hình 16: Sơ đồ tuần tự cho module cảm biến nhiệt độ DS18B20	43
Hình 17: Sơ đồ tuần tự cho module đặt lịch cung cấp thức ăn.....	45
Hình 18: lưu đồ giải thuật mô tả hoạt động của mô hình cho cá ăn tự động	46
Hình 19: Lưu đồ mô tả cơ chế tạo thời gian chuẩn xác bằng ngắt SysTick để điều khiển servo cho ăn 2 giây.	47
Hình 20: Hình ảnh gửi dữ liệu thành công từ STM32 đến ESP32 (8 bit).....	48
Hình 21: Kết nối Wifi và đồng bộ thời gian.....	49
Hình 22: Bộ cụ giao diện của trang web người dùng	51

DANH MỤC BẢNG BIỂU

Bảng 3: Thông số kỹ thuật của Module relay SRD-5VDC-SL-C	24
Bảng 4: Thông số kỹ thuật của module SG90 Servo 180 Độ.....	25

DANH MỤC CÁC CHỮ VIẾT TẮT

IOT	Internet of things
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
TLS	Transport Layer Security
JSON	JavaScript Object Notation
API	Application Programming Interface
MCU	Microcontroller Unit
DB	Database
GUI	Graphical User Interface
FE	Frontend
BE	Backend

MỞ ĐẦU

GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1. Giới thiệu

Nuôi cá cảnh tại Việt Nam đã và đang trở thành một hoạt động phổ biến, không chỉ phục vụ nhu cầu giải trí và thư giãn mà còn mang lại giá trị kinh tế đáng kể cho các địa phương và cộng đồng. Với sự phát triển mạnh mẽ trong những năm qua, nuôi cá cảnh không chỉ dừng lại ở việc làm đẹp không gian sống mà còn góp phần vào việc cải thiện đời sống người dân, tạo ra nguồn thu nhập ổn định. TP.HCM, trung tâm lớn nhất về nuôi và xuất khẩu cá cảnh, đã sản xuất khoảng 110 triệu con cá cảnh trong năm 2023, tăng 17,3% so với năm 2022. Trong đó, 12 triệu con đã được xuất khẩu, mang về 12,5 triệu USD, cho thấy sự phát triển mạnh mẽ và tiềm năng xuất khẩu của ngành cá cảnh tại thành phố này [1]. Đây là một minh chứng rõ ràng cho tiềm năng phát triển của nghề nuôi cá cảnh ở nhiều khu vực khác nhau của Việt Nam. Bên cạnh đó, một nghiên cứu tại TP.HCM cho thấy, 75% người nuôi cá cảnh tự chăm sóc cá của mình, trong khi 13% giao cho người thân chăm sóc và 12% cả gia đình cùng chăm sóc. Điều này cho thấy sự gắn bó, tình yêu và trách nhiệm của người nuôi cá đối với thú chơi này. Về thời gian chăm sóc và ngắm cá, kết quả nghiên cứu cũng chỉ ra rằng 59% người nuôi dành dưới 30 phút/ngày để chăm sóc và theo dõi cá, 28% dành từ 30 đến 60 phút/ngày, và 13% dành hơn 60 phút/ngày cho việc chăm sóc cá cảnh của mình [2]. Như trong nghiên cứu [2] đã chỉ ra kết quả khảo sát cho thấy tỉ lệ người có số năm nuôi từ 1-3 năm chiếm cao nhất khoảng 58,00 %, 4-6 năm chiếm 23,00 %, 7-10 năm chiếm 17,00 %, 11-14 năm và 15-18 năm có cùng tỉ lệ là 1,00 %. Kiểu hồ nuôi cá cảnh phổ biến nhất là bể kiếng đặt trên giá đỡ chiếm tỉ lệ cao nhất 78,64%. Kiểu hồ treo tường chiếm tỉ lệ ít nhất 2,91% trong khi các kiểu hồ khác như hồ non bộ, hồ xi măng cạn, chậu, hũ... chiếm 18,45%.

Từ đây có thể thấy rằng nhu cầu về một hệ thống chăm sóc cá cảnh tự động và thông minh trong gia đình là rất lớn. Để đáp ứng nhu cầu này, nhóm đã phát triển hệ thống giám sát môi trường sống và cung cấp thức ăn tự động trong hồ cá tại nhà như một giải pháp hiệu quả. Hệ thống kết hợp cảm biến siêu âm HCSR04 để cảnh báo khi hết thức ăn, ESP điều khiển các tác vụ như cho ăn theo lịch hay bật/tắt đèn. Người dùng có thể quản lý trực tiếp thông qua giao diện web được xây dựng bằng HTML/CSS (frontend) và Flask (backend), với dữ liệu lưu trữ trên Firebase để đảm bảo tính liên tục và lịch sử

hoạt động. Các tính năng bao gồm đặt lịch cho ăn, theo dõi lịch sử cho ăn qua bảng log, điều khiển ánh sáng, xem video trực tuyến và nhận cảnh báo hết thức ăn. Toàn bộ hệ thống vận hành dựa trên giao thức MQTT, giúp việc truyền nhận dữ liệu nhanh chóng và ổn định. Nhờ đó, người nuôi cá không chỉ tiết kiệm thời gian và công sức, mà còn đảm bảo một môi trường sống ổn định và an toàn cho cá cảnh ngay tại gia đình. Tuy nhiên, việc triển khai hệ thống này cũng đặt ra những thách thức, chẳng hạn như độ ổn định của kết nối Wi-Fi, khả năng bảo mật dữ liệu hay việc mở rộng hệ thống cho nhiều hồ cá khác nhau. Những vấn đề này sẽ được phân tích và thảo luận trong báo cáo, đồng thời đề xuất các giải pháp nhằm tối ưu hóa hiệu quả.

2. Mục tiêu của đề tài

Đề tài hướng đến việc thiết kế và triển khai một mô hình giám sát và chăm sóc cá cảnh tự động. Kiến trúc mô hình gồm có STM32 đảm nhận việc điều khiển thời gian thực đối với cảm biến và cơ cấu chấp hành, trong khi ESP8266 phụ trách kết nối Wi-Fi và giao tiếp IoT. Hệ thống được kỳ vọng sẽ giúp tự động hóa quá trình cho ăn, giám sát lượng thức ăn, điều khiển môi trường sống, đồng thời cung cấp công cụ quản lý trực quan và thuận tiện cho người nuôi, cụ thể:

- Xây dựng chức năng hẹn giờ và điều khiển cho ăn tự động có đèn sáng/tắt.
- Thiết kế mô-đun cảm biến giám sát mức thức ăn tích hợp cơ chế cảnh báo khi gần hết thức ăn và ghi nhận lịch sử tiêu thụ.
- Tích hợp ESP8266 cho truyền dữ liệu và giám sát từ xa, đồng bộ hóa thông tin qua cơ sở dữ liệu và nền tảng ứng dụng, hỗ trợ cấu hình và điều khiển qua mạng.
- Phát triển mô-đun hiển thị thông số đồng thời phân tích số liệu (nhiệt độ, độ đục, mức thức ăn) nhằm nâng cao hiệu quả quản lý.

3. Đối tượng, phạm vi nghiên cứu

❖ Đối tượng nghiên cứu: Các loài cá cảnh nước ngọt nuôi trong hồ tại nhà:

- Cá Molly (*Poecilia spp.*) là loài ăn tạp thiên về thực vật, thích hợp trong bể cộng đồng. Trong hồ $50 \times 27 \times 30$ cm, nên cho ăn lượng vừa đủ để cá tiêu thụ hết trong 1–2 phút, từ 1–3 lần/ngày. Trung bình ước lượng thức ăn 5g/ngày
- Cá Neon tetra (*Paracheirodon innesi*) là loài cá nhỏ bơi đàn, cần khẩu phần rất ít. Mỗi lần chỉ cho ăn lượng vừa đủ trong 1–2 phút, tần suất 1–2 lần/ngày. Thức ăn phù hợp là thức ăn cho cá cảnh và ước lượng trung bình 1g/ngày

- Cá Vàng (*Carassius auratus*) có tập tính ăn nhiều, dễ gây bẩn nước nên cần kiểm soát khẩu phần. Cho ăn vừa đủ trong vòng 2 phút, 1 lần/ngày. Thức ăn chính là viên dành cho cá vàng. Trung bình ước lượng thức ăn 50g/ngày
- Cá lau kính (Bristlenose Pleco, *Ancistrus*) là loài ăn đáy thiên về thực vật, thường ăn vào ban đêm. Cho lượng algae wafer hoặc rau củ mà cá ăn hết trong khoảng 2 giờ, tần suất 1 lần buổi tối (có thể tăng nếu ít tảo). Thức ăn chủ yếu là tảo và rau củ, thỉnh thoảng bổ sung đạm, đồng thời cần có gỗ lũa để cá gặm lấy xơ.

❖ Phạm vi nghiên cứu:

- Kiến trúc điều khiển: STM32, ESP8266
- Độ chính xác đo:
 - Nhiệt độ: sai số mục tiêu $\leq \pm 0,5$ °C sau hiệu chuẩn đa điểm.
 - Độ đục: độ lặp lại (SD) $\leq 5\%$ trong dải quan tâm; thiết lập đường cong ADC $\rightarrow$ NTU tối thiểu 5 điểm.
 - Định lượng cho ăn: sai số mục tiêu $\leq \pm 10\%$ so với giá trị gram tham chiếu (cân thật), ăn hết trong ≤ 2 phút.
 - Độ tin cậy vận hành: uptime $\geq 90\%$ /tuần; mất Wi-Fi hệ thống vẫn cho ăn theo lịch cục bộ; độ trễ MQTT trong LAN mục tiêu ≤ 1 s.
- Môi trường thực nghiệm: Hồ cá tại nhà kính kích thước $50 \times 27 \times 30$ cm (khoảng 40 lít) và thức ăn khô dạng mảnh/viên cho cá cảnh

4. Phương pháp nghiên cứu

❖ Phương pháp thu thập số liệu:

- Sử dụng phương pháp quan sát: Nhóm tiến hành quan sát trực tiếp các yếu tố tác động đến chất lượng nước, môi trường sống của cá và quá trình chăm sóc cá trong các hồ cá thực tế. Qua việc quan sát, nhóm thu thập thông tin về các yếu tố như độ đục nước, nhiệt độ nước, và các thói quen chăm sóc cá của người nuôi.
- Tiến hành phỏng vấn: Nhóm thực hiện các cuộc phỏng vấn với người nuôi cá và các chuyên gia trong ngành để hiểu rõ yêu cầu và kỳ vọng của họ đối với mô hình giám sát và chăm sóc cá tự động. Thông qua phỏng vấn, nhóm thu thập thông tin về các vấn đề mà người nuôi đang gặp phải.

❖ Phương pháp thực nghiệm

- Thiết kế và triển khai hệ thống: Tôi tiến hành thiết kế và triển khai một mô hình quan sát và cho cá ăn tự động
- Kiểm thử: lịch cho ăn, cảnh báo hết thức ăn, phát hiện kẹt motor/timeout, mất điện/mạng và khởi động lại.
- Đánh giá hiệu quả: Sau khi triển khai mô hình, mô hình tiến hành đánh giá hiệu quả của mô hình dựa trên các tiêu chí như độ chính xác của cảm biến, hiệu quả cảnh báo và khả năng tự động hóa các tác vụ chăm sóc cá.

❖ Phương pháp phân tích – tổng kết

- Phân tích dữ liệu: Nhóm phân tích dữ liệu thu thập từ các cảm biến và mô hình giám sát để đánh giá độ tin cậy và tính chính xác của thông tin về môi trường nước, trạng thái của cá và các yếu tố chăm sóc.
- Tổng kết kinh nghiệm: Dựa trên kết quả phân tích, nhóm tổng kết kinh nghiệm từ quá trình triển khai mô hình. Đánh giá các ưu điểm và hạn chế của mô hình, đồng thời đề xuất các giải pháp và khuyến nghị để cải thiện hiệu quả hoạt động của mô hình giám sát và chăm sóc cá tự động.

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT

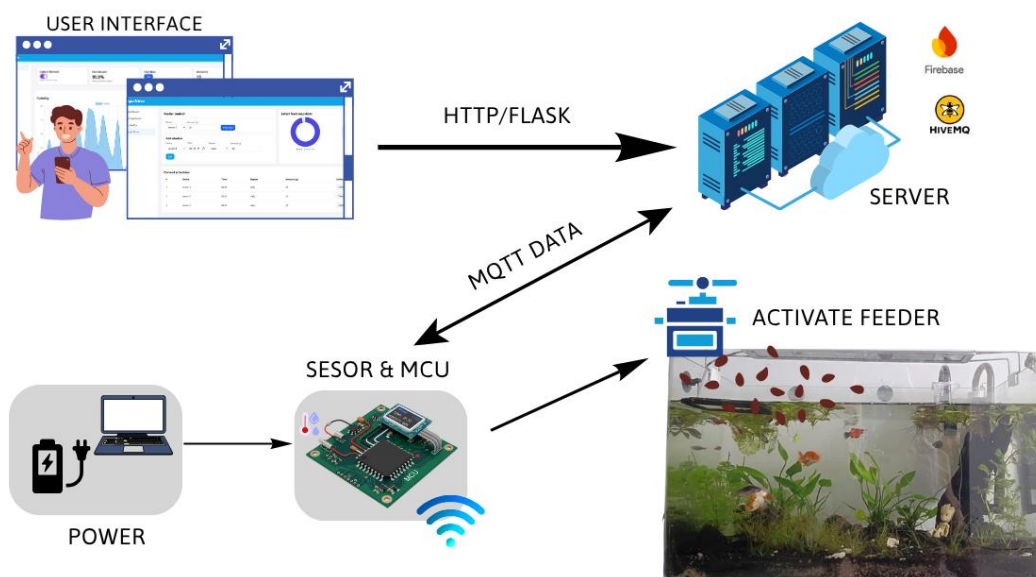
1.1. Mô hình giám sát chất lượng nước và cung cấp thức ăn tự động

1.1.1. Mô hình giám sát chất lượng nước, độ đục và cho cá ăn

Mô hình giám sát chất lượng nước trong bể cá được thiết kế nhằm theo dõi các thông số môi trường và tự động hóa quá trình chăm sóc. Hệ thống sử dụng các cảm biến đo nhiệt độ, độ đục của nước, sau đó truyền dữ liệu về vi điều khiển trung tâm ESP8266. Thông qua kết nối Wi-Fi tích hợp, ESP8266 gửi dữ liệu đến máy chủ để lưu trữ và xử lý. Tại đây, thông tin được trực quan hóa dưới dạng biểu đồ và bảng số liệu, đồng thời hệ thống có khả năng phát cảnh báo khi thông số vượt ngưỡng cho phép.

Ngoài chức năng giám sát, mô hình còn được tích hợp cơ cấu chấp hành điều khiển bộ cho ăn tự động thông qua relay. Người sử dụng có thể cài đặt lịch cho ăn hoặc kích hoạt từ xa thông qua ứng dụng trên thiết bị di động. Điều này giúp duy trì chế độ chăm sóc ổn định, giảm thiểu sự phụ thuộc vào sự có mặt trực tiếp của người nuôi, đồng thời nâng cao hiệu quả quản lý.

Luồng hoạt động tổng thể của mô hình có thể được mô tả như sau: dữ liệu được cảm biến thu thập và truyền về ESP8266 → dữ liệu được gửi đến máy chủ để xử lý và hiển thị → người sử dụng theo dõi thông tin, nhận cảnh báo và đưa ra lệnh điều khiển từ xa → hệ thống kích hoạt bộ cho ăn thông qua relay khi nhận tín hiệu điều khiển. Với cấu trúc này, mô hình đảm bảo tính tự động, khả năng giám sát theo thời gian thực và sự tương tác linh hoạt giữa người sử dụng và hệ thống.

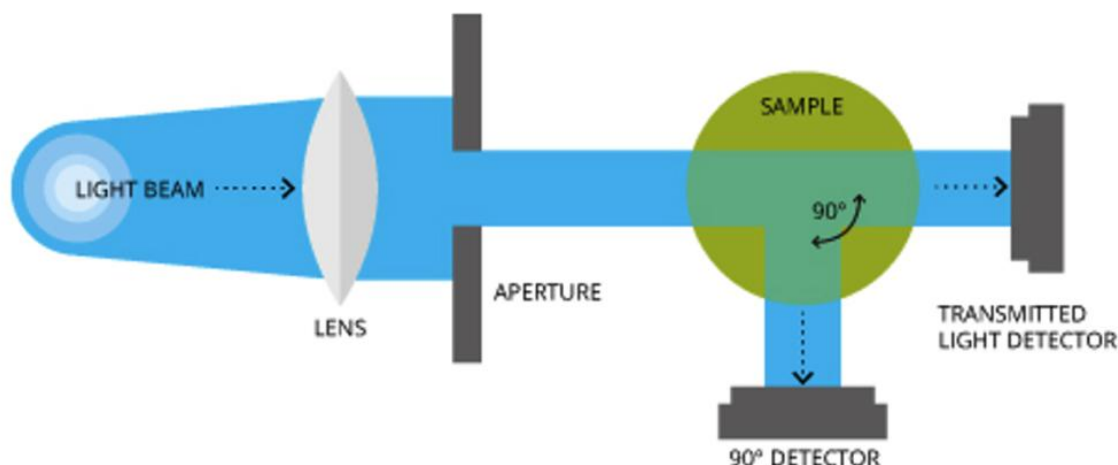


Hình 1: Tổng quan mô hình giám sát chất lượng nước (nhiệt độ, độ đục) và điều khiển cho ăn tự động thông qua kết nối Internet. Dữ liệu cảm biến được gửi từ ESP32/MCU lên server Flask qua MQTT, sau đó lưu trữ trên Firebase và hiển thị trên giao diện web. Người dùng đặt lịch hoặc kích hoạt cho ăn trực tiếp từ dashboard, lệnh được gửi ngược về thiết bị để điều khiển cơ cấu cho ăn.

1.1.2. Nguyên lý đo độ đục của nước

Độ đục của nước là một chỉ tiêu quan trọng trong đánh giá chất lượng môi trường nước, thường được biểu thị qua các đơn vị NTU (Nephelometric Turbidity Units), FNU (Formazin Nephelometric Units), FTU (Formazin Turbidity Units) và FAU (Formazin Attenuation Units). Các đơn vị này được xem là tương đương nhau, tức $1 \text{ NTU} = 1 \text{ FNU} = 1 \text{ FTU} = 1 \text{ FAU}$ [3]. Độ đục bắt nguồn từ sự hiện diện của các hạt lơ lửng trong nước như tảo, bụi bẩn, khoáng chất, dầu, vi khuẩn và nhiều hợp chất khác, khiến nước mất đi tính trong suốt và thay đổi màu sắc. Cần phân biệt độ đục với tổng chất rắn hòa tan (TDS – Total Dissolved Solids). Trong khi độ đục phản ánh khả năng truyền sáng của nước thông qua hiện tượng tán xạ và hấp thụ ánh sáng, thì TDS đo tổng khối lượng các chất hòa tan. Do vậy, độ đục không chỉ là biểu hiện trực quan về sự “trong” hay “đục”, mà còn phản ánh những yếu tố vật lý, hóa học và sinh học liên quan đến chất lượng nước [4]. Nước có độ đục cao thường hấp thụ nhiều nhiệt, dẫn đến tăng nhiệt độ và làm giảm nồng độ oxy hòa tan. Đồng thời, lớp nước đục hạn chế ánh sáng xuyên qua, gây cản trở quang hợp và làm suy giảm khả năng sản sinh oxy của hệ sinh thái thủy sinh [5]. Điều này ảnh hưởng trực tiếp đến sự sống và phát triển của nhiều loài thủy sinh. Các nguyên nhân gây độ đục rất đa dạng, bao gồm xói mòn đất, dòng chảy đô thị, hoạt động nông nghiệp, xả thải công nghiệp và các quá trình tự nhiên. Hiện nay, phương pháp đo độ đục

phổ biến là Nephelometry dựa trên nguyên lý tán xạ ánh sáng. Theo tiêu chuẩn ISO 7027, độ đục được xác định bằng cách chiếu chùm sáng đơn sắc (thường ~ 860 nm) vào mẫu nước và đo cường độ ánh sáng tán xạ ở góc 90° so với nguồn phát. Cách bố trí này giúp giảm ảnh hưởng của màu sắc nước và tăng độ chính xác trong phép đo [6], [7].



Hình 2: Sơ đồ minh họa nguyên lý hoạt động của cảm biến độ đục theo tán xạ ánh sáng. Hiện thị LED phát sáng $\rightarrow$ mẫu nước $\rightarrow$ detector góc 90° thu ánh sáng tán xạ [8].

1.1.3. Hình thức cung cấp thức ăn

Thiết bị được thiết kế dựa trên sự kết hợp giữa một chai nhựa đặt ngược, một ống dẫn, một trục xoắn, một động cơ servo quay 180° và một cảm biến siêu âm. Chai nhựa đóng vai trò như khoang chứa chính, có thể dễ dàng tháo rời để bổ sung thức ăn và được lắp ngược phía trên cùng của hộp. Nắp chai được nối liền với ống dẫn đặt ở bên dưới, giúp hạt thức ăn có thể rơi tự do theo trọng lực mà không bị kẹt. Bên trong ống dẫn có bố trí trục xoắn được gắn trực tiếp với servo. Khi servo được vi điều khiển kích hoạt, trục xoắn sẽ quay với tốc độ chậm, tạo ra cơ chế đẩy thức ăn từ từ ra phía cửa xả. Thức ăn sau đó đẩy ra khay nhỏ ở phía ngoài, rồi tiếp tục rơi xuống bể cá, bảo đảm lượng thức ăn được phân phối đều đặn và tránh hiện tượng cá ăn quá nhiều trong một lần.

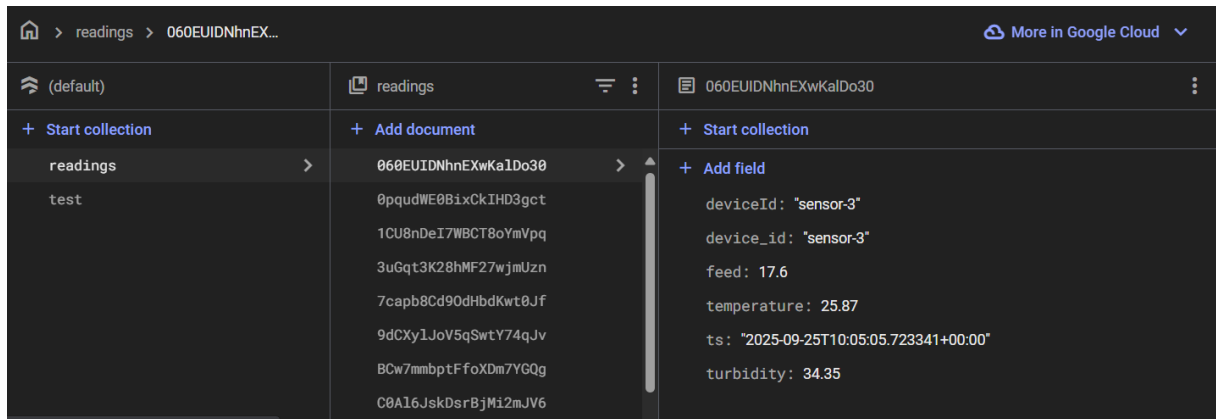
Điểm nổi bật của thiết kế này nằm ở việc tích hợp thêm cảm biến siêu âm đặt tại nắp chai. Cảm biến có nhiệm vụ đo khoảng cách bên trong khoang chứa để xác định lượng thức ăn còn lại, từ đó gửi tín hiệu về vi điều khiển nhằm đưa ra cảnh báo hoặc hiển thị thông tin trên ứng dụng khi thức ăn gần hết.

Cơ chế cung cấp thức ăn của thiết bị vừa đảm bảo tính tự động hóa, vừa chú trọng đến độ bền và sự tiện dụng trong quá trình sử dụng. Nhờ sự kết hợp giữa chai nhựa chứa

thức ăn, ống dẫn, trục xoắn và servo quay 180 độ, thức ăn được phân phối đều đặn, kiểm soát và hạn chế tình trạng cá ăn quá mức. Việc tích hợp cảm biến siêu âm ở khoang chứa giúp người dùng luôn nắm rõ lượng thức ăn còn lại và nhận cảnh báo kịp thời khi gần hết, tạo sự an tâm trong quá trình chăm sóc cá.

1.2. Cơ sở dữ liệu

1.2.1. Firsebase



Cloud Firestore là cơ sở dữ liệu NoSQL linh hoạt và có khả năng mở rộng, được thiết kế để lưu trữ và đồng bộ dữ liệu giữa các ứng dụng khách và máy chủ theo thời gian thực. Firestore tổ chức dữ liệu dưới dạng collection và document, cho phép truy vấn mạnh mẽ, cập nhật tức thời và hỗ trợ hoạt động ngoại tuyến. Ngoài ra, Firestore còn tích hợp cơ chế bảo mật thông qua Firebase Security Rules và khả năng mở rộng toàn cầu trên hạ tầng Google Cloud[9]. Một ưu điểm quan trọng của Firestore là khả năng đồng bộ thời gian thực, cho phép dữ liệu được cập nhật ngay lập tức trên tất cả các thiết bị khi có thay đổi. Hệ thống có khả năng mở rộng mạnh mẽ, hỗ trợ xử lý khối lượng lớn dữ liệu và người dùng đồng thời. Bên cạnh đó, Firestore cung cấp cơ chế truy vấn đa dạng như lọc, sắp xếp và phân trang dữ liệu, đáp ứng tốt nhu cầu khai thác dữ liệu trong các ứng dụng IoT. Về bảo mật, Firestore hỗ trợ Firebase Security Rules ở mức tài liệu và tập hợp, cho phép kiểm soát truy cập chi tiết và an toàn. Ngoài ra, Firestore còn có độ tin cậy cao nhờ multi-region replication, giúp đảm bảo tính sẵn sàng và hạn chế nguy cơ mất mát dữ liệu.

1.3. UI (User interface) và UX (User experience)

1.3.1. HTML (HyperText Markup Language)

HTML (HyperText Markup Language) là một ngôn ngữ đánh dấu được sử dụng như một công cụ giúp xây dựng cấu trúc và định dạng nội dung trên trang web. HTML là ngôn ngữ căn bản và chủ yếu sử dụng các thẻ (tag) để xác định các phần tử và sắp xếp chúng thành các thành phần như tiêu đề, đoạn văn bản, hình ảnh, liên kết và bảng.

Cấu trúc cơ bản của HTML: Một tài liệu HTML sẽ luôn bắt đầu bằng thẻ `<html>` và kết thúc cũng bằng thẻ `</html>`. Trong tài liệu HTML, chúng ta sẽ thấy các phần tử như `<head>`, `<body>`, `<title>`, `<h1>`, `<p>`, `<img>`, `<a>`, `<table>`, `<form>` và nhiều phần tử khác. HTML cung cấp các thẻ để định dạng nội dung như chữ in đậm (`<strong>`), chữ in nghiêng (`<em>`), danh sách (`<ul>`, `<ol>`, `<li>`), và định dạng văn bản (`<h1>`, `<p>`, `<span>`) để làm nổi bật nội dung và cung cấp cấu trúc cho trang web. Ngoài ra, HTML còn cho phép chúng ta tạo liên kết đến các nơi khác bằng cách sử dụng thẻ `<a>` và tạo hình ảnh bằng cách sử dụng thẻ `<img>`. Chúng ta có thể định rõ đường dẫn, mô tả và thuộc tính khác cho các liên kết và hình ảnh này. Cuối cùng, nhằm mục đích trực quan hóa dữ liệu một cách tốt hơn, HTML hỗ trợ việc tạo bảng dữ liệu nhờ các thẻ `<table>`, `<tr>`, `<td>`, và các thẻ liên quan. Chúng ta cũng có thể tạo biểu đồ đơn giản bằng cách sử dụng các phần tử HTML và CSS.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Tiêu đề trang</title>
</head>
<body>
  <h1>Đây là tiêu đề chính</h1>
  <p>Đây là một đoạn văn bản trong HTML.</p>
  
  <a href="https://www.example.com">Đây là một liên kết</a>
</body>
</html>
```

1.3.2. CSS (Cascading Style Sheets)

CSS (Cascading Style Sheets) là ngôn ngữ định kiểu được sử dụng để mô tả cách trình bày và hiển thị của các phần tử trong tài liệu HTML. Nếu HTML quy định cấu trúc

nội dung của một trang web thì CSS đóng vai trò như lớp “giao diện” phủ lên, giúp điều chỉnh màu sắc, kích thước, khoảng cách, phong chữ, hiệu ứng và bố cục, từ đó mang lại trải nghiệm trực quan và thẩm mỹ hơn cho người dùng.

Lựa chọn phần tử: CSS sử dụng các bộ chọn (selectors) để xác định phần tử mà chúng ta muốn kiểm soát. Các bộ chọn có thể là tên thẻ HTML (h1, p), lớp (class), ID (id), thuộc tính ([attribute]), và nhiều lựa chọn khác.

Thuộc tính và giá trị: CSS cho phép chúng ta thiết lập các thuộc tính như color, font size, margin, padding, background-color, border, text-align, và nhiều thuộc tính khác. Chúng ta có thể thiết lập giá trị của thuộc tính để định dạng và kiểm soát hiển thị của các phần tử HTML.

Lớp và ID: CSS hỗ trợ việc định nghĩa lớp (class) và ID (id) cho các phần tử HTML, giúp chúng ta áp dụng kiểu dáng chỉ định cho từng phần tử cụ thể hoặc nhóm phần tử. Các phương pháp kiểu dáng: CSS hỗ trợ các phương pháp kiểu dáng bao gồm Inline, Internal và External. Inline CSS áp dụng kiểu dáng trực tiếp vào các phần tử HTML, Internal CSS được định nghĩa trong thẻ <style> trong tài liệu HTML, và External CSS được định nghĩa trong một tệp riêng biệt và được liên kết tài liệu HTML.

```
<p style="color: red;">Văn bản màu đỏ</p>
<style>
  h1 {
    color: green
    font-size: 24px;
  }
  p { color: blue; }
</style>
<link rel="stylesheet" href="style.css">
```

1.3.3. JavaScript (JS)

JavaScript (JS) là ngôn ngữ lập trình được sử dụng rộng rãi trong phát triển web, cho phép các trang web trở nên tương tác và động. JS có thể thay đổi nội dung HTML, xử lý sự kiện người dùng, và giao tiếp với server để cập nhật dữ liệu mà không cần tải lại trang. Trong hệ thống hồ cá thông minh, JavaScript được dùng để hiển thị dữ liệu cảm biến thời gian thực và gửi lệnh điều khiển đến server thông qua các yêu cầu HTTP. Thông qua các phương thức HTTP như GET và POST, JavaScript có thể gửi và nhận dữ liệu với server Flask mà không cần tải lại trang, giúp việc điều khiển và giám sát hệ thống trở nên mượt mà hơn. Trong ứng dụng mô hình giám sát hồ cá, JS sử dụng Fetch API để trao đổi dữ liệu:

- Phương thức GET: dùng để truy vấn và nhận dữ liệu từ server (ví dụ: lấy danh sách lịch cho ăn hoặc dữ liệu cảm biến).

- Phương thức POST: dùng để gửi dữ liệu mới hoặc yêu cầu điều khiển (ví dụ: thêm lịch, cho ăn ngay).

Ví dụ mã JavaScript cơ bản gửi yêu cầu đến server bằng phương thức POST: Đoạn mã trên minh họa cách JS sử dụng Fetch API để gửi dữ liệu dạng JSON đến server và nhận phản hồi trả về, giúp tạo nên cầu nối giữa giao diện người dùng và hệ thống backend.

```
// Gửi dữ liệu đến server Flask qua HTTP POST
fetch("/control/feed-now", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    device_id: "esp32-feeder-1",
    amount: 30
  })
})
.then(response => response.json())
.then(data => console.log("Server response:", data))
.catch(error => console.error("Error:", error));
```

1.3.4. Flask (micro web framework for Python)

Flask là một micro web framework được xây dựng trên ngôn ngữ Python, thường được sử dụng để phát triển các ứng dụng web và dịch vụ API một cách nhanh chóng.

Đặc điểm chính:

- Nhẹ và đơn giản: Flask được thiết kế với kiến trúc tối giản, chỉ cung cấp những thành phần cốt lõi cần thiết (routing, template engine, HTTP handling).
- Mở rộng dễ dàng: có thể tích hợp thêm nhiều thư viện/phần mở rộng như quản lý cơ sở dữ liệu (SQLAlchemy), xác thực người dùng, gửi mail, hoặc API RESTful.
- Routing linh hoạt: dễ dàng định nghĩa các đường dẫn (URL) và ánh xạ tới các hàm xử lý trong ứng dụng.
- Hỗ trợ Jinja2 template: cho phép tạo giao diện HTML động kết hợp dữ liệu từ server.
- Thân thiện với REST API: thường được dùng để triển khai backend cho ứng dụng IoT, mobile, hoặc web vì có thể trả về dữ liệu JSON nhanh gọn.

Ứng dụng trong hệ thống IoT: Flask đóng vai trò là máy chủ web (Web Server):

- Nhận dữ liệu từ MQTT và lưu trữ vào cơ sở dữ liệu (Firebase Firestore).
- Cung cấp API để dashboard hoặc ứng dụng web/mobile truy xuất dữ liệu cảm biến.
- Xử lý lệnh điều khiển (bật/tắt, cho ăn, lên lịch) và gửi ngược lại cho thiết bị thông qua MQTT Broker.

1.4. Các chuẩn truyền thông

1.4.1. Các chuẩn truyền thông không dây

Chuẩn truyền IEEE 802.11, hay còn được gọi là WiFi, là một chuẩn truyền không dây phổ biến được sử dụng để truyền dữ liệu qua mạng local area network (LAN). Được phát triển bởi Hiệp hội Kỹ sư Điện và Điện tử (IEEE), chuẩn truyền này cho phép truyền dữ liệu giữa các thiết bị mạng không dây như máy tính, điện thoại di động, máy tính bảng và các thiết bị kết nối Internet khác.

Các phiên bản của chuẩn truyền IEEE 802.11 WiFi:

- + IEEE 802.11a: Được phát triển vào năm 1999 và hoạt động trong dải tần 5GHz, cung cấp tốc độ truyền dữ liệu lên đến 54Mbps.
- + IEEE 802.11b: Được phát triển cùng thời với IEEE 802.11a, hoạt động trong dải tần 2.4GHz cung cấp tốc độ truyền dữ liệu lên đến 11Mbps.
- + IEEE 802.11g: Được phát triển vào năm 2003 và hoạt động trong cả dải tần 2.4GHz cung cấp tốc độ truyền dữ liệu lên đến 54Mbps và sử dụng kỹ thuật OFDM như IEEE 802.11a.
- + IEEE 802.11n: Được phát triển vào năm 2009 hoạt động trong cả dải tần 2.4GHz và 5GHz và cung cấp tốc độ truyền dữ liệu lên đến 600Mbps.
- + IEEE 802.11ac: Được phát triển vào năm 2013, hoạt động trong dải tần 5GHz và cung cấp tốc độ truyền dữ liệu lên đến hàng trăm Mbps hoặc thậm chí hàng ngàn Mbps.

Các đặc điểm của chuẩn truyền IEEE 802.11 WiFi:

- Tốc độ truyền dữ liệu cao.
- Phổ sóng rộng và xuyên vật cản
- Độ bảo mật cao.
- Tương thích ngược với các phiên bản cũ.

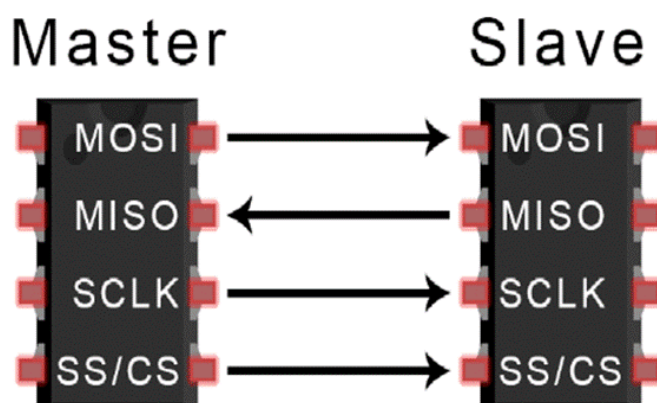
1.4.2. Các chuẩn truyền thông có dây

a) Chuẩn truyền thông SPI

Chuẩn truyền SPI (Serial Peripheral Interface) là một giao thức truyền thông đồng bộ (synchronous) được sử dụng để kết nối và truyền dữ liệu giữa các thiết bị điện tử trong hệ thống nhúng (embedded systems). Nó được phát triển bởi Motorola vào những năm 1980 và sau đó trở thành một tiêu chuẩn ngành công nghiệp.

SPI bao gồm hai thành phần chính: Master (thiết bị chủ) và Slave (thiết bị con). Master là nguồn điều khiển gửi lệnh và nhận dữ liệu từ Slave. Các Slave là các thiết bị nhận và

trả lời các lệnh từ Master. SPI sử dụng hai đường truyền chính. Master gửi dữ liệu tới Slave thông qua đường truyền dữ liệu (MOSI - Master Output Slave Input). Slave trả lời Master thông qua đường truyền dữ liệu (MISO - Master Input Slave Output)



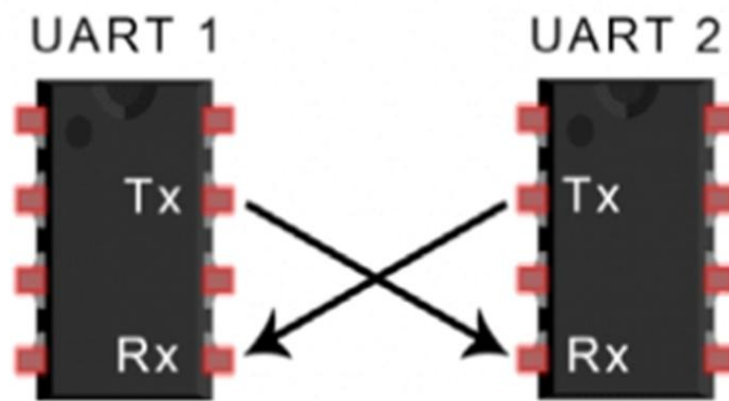
Hình 3: Cách kết nối của chuyển truyền SPI.

Tốc độ truyền dữ liệu: SPI cho phép truyền dữ liệu với tốc độ cao. Tuy nhiên, tốc độ truyền dữ liệu phụ thuộc vào tốc độ đồng hồ và cấu hình của hệ thống. SPI không yêu cầu giao thức truyền cụ thể. Nó chỉ cung cấp cơ chế truyền dữ liệu đồng bộ giữa Master và Slave. SPI cho phép cấu hình các tham số như tốc độ truyền, kiểu truyền (full-duplex, half-duplex) và thứ tự truyền (LSB-first, MSB-first). SPI có tính tương thích ngược, cho phép kết nối và giao tiếp giữa các thiết bị SPI từ các nhà sản xuất khác nhau.

b) Chuẩn truyền UART

Chuẩn truyền UART (Universal Asynchronous Receiver-Transmitter) là một giao thức truyền thông bất đồng bộ (asynchronous) được sử dụng trong việc kết nối và truyền dữ liệu giữa các thiết bị điện tử. Giao thức UART cho phép truyền dữ liệu qua một đường truyền đơn (simplex) hoặc hai đường truyền song song (duplex).

Giao thức UART sử dụng cơ chế truyền dữ liệu không đồng bộ, không yêu cầu một tín hiệu đồng hồ chung giữa các thiết bị. Nó dựa trên việc sử dụng một chuỗi các bit để truyền dữ liệu. Các bit được truyền theo thứ tự từ bit thấp đến bit cao (LSB first) hoặc từ bit cao đến bit thấp (MSB first). Giao thức UART bao gồm hai tín hiệu chính: Tín hiệu truyền (TX - Transmitter) và tín hiệu nhận (RX - Receiver). Thiết bị gửi dữ liệu sẽ sử dụng tín hiệu này để truyền dữ liệu đến thiết bị nhận. Thiết bị nhận dữ liệu sẽ sử dụng tín hiệu này để nhận dữ liệu từ thiết bị gửi.



Hình 4: Cách kết nối của chuẩn truyền UART.

Tốc độ truyền dữ liệu của giao thức UART được xác định bằng số lượng bit truyền trong một giây (baud rate). Các tốc độ phổ biến bao gồm 9600, 19200, 38400, 115200, và nhiều tốc độ khác. Khả năng truyền dữ liệu không đồng bộ: Giao thức UART cho phép truyền dữ liệu không đồng bộ, nghĩa là không cần một tín hiệu đồng hồ chung giữa các thiết bị. Chuẩn truyền UART có tính tương thích ngược, cho phép kết nối cũng như giao tiếp giữa các thiết bị UART từ các nhà sản xuất khác nhau. Giao thức UART là giao thức truyền thông đơn giản và phổ biến trong ứng dụng nhúng và vi điều khiển.

1.4.3. Giao thức IoT: MQTT

MQTT là một giao thức nhắn tin theo mô hình xuất bản/đăng ký (Publish/Subscribe), được thiết kế để trở nên nhẹ (lightweight), phù hợp với các thiết bị bị hạn chế tài nguyên (bộ nhớ, CPU) và các mạng có băng thông thấp, độ trễ cao hoặc không ổn định. Đây là một giao thức lý tưởng cho Internet Vạn Vật (IoT) và giao tiếp Machine-to-Machine (M2M). Ngoài MQTT COAP là một giao thức IoT phổ biến. Cả hai đều được thiết kế để truyền dữ liệu nhẹ, phù hợp với các thiết bị nhúng có tài nguyên hạn chế. Tuy nhiên, chúng khác nhau về kiến trúc, cơ chế đảm bảo truyền tin và mô hình kết nối.

Tiêu chí	MQTT	CoAP
Mô hình giao tiếp	Publish/Subscribe thông qua Broker trung gian	Client/Server trực tiếp
Giao thức nền	TCP/IP (đảm bảo kết nối)	UDP (nhẹ hơn nhưng không đảm bảo)
Cơ chế tin cậy	Hỗ trợ QoS (0, 1, 2) xác nhận truyền tin	Có ACK nhưng phụ thuộc vào UDP
Cách tổ chức hệ thống	Các thiết bị gửi dữ liệu lên Broker, bên khác đăng ký (subscribe) để nhận	Thiết bị gửi yêu cầu GET/POST đến server

Bảo mật	Hỗ trợ SSL/TLS, xác thực người dùng	Hỗ trợ DTLS, cấu hình phức tạp hơn
Độ trễ	Trung bình (do TCP)	Rất thấp (UDP)
Khả năng mở rộng	Cao, chỉ cần thêm topic	Hạn chế, cần thiết lập từng kết nối
Phù hợp cho	Hệ thống lớn, nhiều node, truyền dữ liệu thường xuyên	Hệ thống cảm biến nhỏ, đơn giản

Mô hình Publish/Subscribe: MQTT hoạt động dựa trên ba thành phần chính:

- Publisher (Bên Xuất bản/Client): Thiết bị hoặc ứng dụng gửi dữ liệu.
- Subscriber (Bên Đăng ký/Client): Thiết bị hoặc ứng dụng nhận dữ liệu.
- Broker (Máy chủ/Server): Thành phần trung gian, nhận thông điệp từ Publisher và chuyển tiếp đến các Subscriber đã đăng ký (subscribe) một Topic cụ thể. Broker giúp tách biệt bên gửi và bên nhận về mặt thời gian và không gian.

Lý do chọn : Lý do chọn giao thức MQTT trong hệ thống IoT cho cá ăn tự động

- Nhẹ và tối ưu cho IoT: MQTT sử dụng băng thông thấp, overhead nhỏ, phù hợp cho các thiết bị tài nguyên hạn chế như STM32 và ESP32.
- Thích hợp cho môi trường có kết nối Wi-Fi không ổn định.
- Mô hình Publish/Subscribe đơn giản: Dữ liệu cảm biến (nhiệt độ, độ đục, lượng thức ăn) từ ESP32 chỉ cần publish một lần. Nhiều subscriber (web dashboard, app di động, server Firebase) đồng thời nhận dữ liệu mà không cần MCU gửi riêng cho từng thiết bị.
- Độ tin cậy cao: Hỗ trợ các mức QoS (Quality of Service) 0, 1, 2 cho phép lựa chọn giữa tốc độ và độ đảm bảo truyền dữ liệu. Có cơ chế Last Will Message và KeepAlive để xử lý mất kết nối.
- Khả năng mở rộng tốt MQTT broker (như HiveMQ Cloud) có thể quản lý hàng ngàn thiết bị cùng lúc. Hệ thống dễ dàng mở rộng thêm cảm biến (nhiệt độ, pH, oxy hòa tan) hoặc actuator (máy bơm, quạt oxy) chỉ bằng cách thêm topic mới.
- Tích hợp thuận tiện với nền tảng đám mây: MQTT có sẵn plugin, bridge kết nối với Firebase, Node-RED, InfluxDB, Grafana, giúp dễ dàng lưu trữ, trực quan hóa và điều khiển từ xa.
- Bảo mật tốt: Hỗ trợ xác thực Username/Password, kết nối TLS/SSL để bảo vệ dữ liệu trong môi trường mạng công cộng (Wi-Fi).

a) Mức Chất lượng Dịch vụ (Quality of Service - QoS)

MQTT cung cấp ba mức QoS để đảm bảo độ tin cậy của việc truyền thông điệp:

QoS 0: At most once (Tối đa một lần)

- Thông điệp được gửi một lần, không có xác nhận (ACK).
- Truyền tải nhanh nhất nhưng có thể bị mất.
- Tương đương: Gửi bưu thiếp thông thường.

QoS 1: At least once (Tối thiểu một lần)

- Thông điệp được đảm bảo đến đích ít nhất một lần. Bên nhận sẽ gửi gói PUBACK xác nhận.
- Thông điệp có thể bị trùng lặp (nếu bên gửi không nhận được PUBACK và gửi lại).
- Tương đương: Gửi thư bảo đảm có biên lai xác nhận.

QoS 2: Exactly once (Duy nhất một lần)

- Thông điệp được đảm bảo đến đích đúng một lần. Sử dụng giao thức bắt tay bốn bước (handshake) để đảm bảo không bị mất hoặc trùng lặp.
- Đáng tin cậy nhất nhưng tốn nhiều băng thông và thời gian nhất.
 - Tương đương: Gửi thư bảo đảm yêu cầu chữ ký và xác nhận giao hàng chi tiết.

QoS 0: phù hợp cho dữ liệu không quan trọng, như cập nhật liên tục nhiệt độ mỗi giây (chấp nhận mất gói).

QoS 1: phù hợp cho dữ liệu quan trọng vừa phải, như trạng thái cho ăn (nếu trùng lặp vẫn không sao).

QoS 2: dùng cho dữ liệu cực kỳ quan trọng, như giao dịch tài chính, lệnh điều khiển chỉ được thực hiện một lần duy nhất.

b) MQTT Connection

Luồng gói tin MQTT

CONNECT: Client gửi đến Broker để yêu cầu thiết lập kết nối.

CONNACK: Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay kết nối thất bại.

```
[sensor-1] log: Sending CONNECT (u1, p1, wr0, wq0, wf0, c1, k60)
client_id=b'sim-sensor-1'
[sensor-1] log: Received CONNACK (0, 0)
[sensor-1] Connected with result code Success
```

- u1, p1: username & password
- c1 : cleanSession = 1 | wq0: mức QoS

- k60: keepAlive = 60 giây
- client_id= 'sim-sensor-1': định danh client
- wf0: khai báo Last Will
- wr0: cờ retain Last Will message.

Sau đó broker trả về CONNACK (0,0) → thành công.

Gói tin	Mô tả	Cấu trúc (Header + Payload)
CONNECT	Client gửi đến Broker để yêu cầu thiết lập kết nối.	Fixed Header + Variable Header (Tên và phiên bản Protocol, cờ kết nối, Keep Alive) + Payload (Client ID, Username/Password, Last Will & Testament)
CONNACK	Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay không	Fixed Header + Variable Header (cờ xác nhận, lý do)
DISCONNECT	Client gửi đến Broker để ngắt kết nối một cách an toàn.	Chỉ có Fixed Header
PINGREQ / PINGRESP	Client gửi PINGREQ để giữ kết nối hoạt động khi không có dữ liệu trao đổi; Broker phản hồi bằng PINGRESP.	Chỉ có Fixed Header

c) MQTT Publisher

Luồng gói tin MQTT

PUBLISH: Thiết bị gửi dữ liệu cảm biến (telemetry) lên topic định sẵn.

PUBACK: Broker xác nhận đã nhận dữ liệu (QoS=1).

Broker trả PUBACK (mid=1) → nhận gói tin thành công

DISCONNECT: Client chủ động ngắt kết nối.

[sensor-1] log: Sending PUBLISH (d0, q1, r0, m1),
 "b'aquanova/devices/sensor-1/telemetry", ... (123 bytes)

```
[sensor-1] -> publishing {'device_id': 'sensor-1', 'ts': '2025-09-30T10:01:05.944232+00:00',
'temperature': 30.53, 'turbidity': 48.55, 'feed': 51.6}
```

- d0: dupFlag = 0 | r0: retain = 0
- q1: QoS 1 (phải có PUBACK xác nhận)
- m1: messageId = 1
- Topic: aquanova/devices/sensor-1/telemetry
- Payload: JSON chứa thông số sensor

```
2025-09-30T17:14:44.996069 [sensor-1] log: Received PUBACK (Mid: 1)
2025-09-30T17:14:44.996898 [sensor-1] Published mid=1, reason=Success
2025-09-30T17:15:34.852617 [sensor-1] log: Sending DISCONNECT
2025-09-30T17:15:34.855632 [sensor-1] log: ... Disconnected, reason=0
```

Gói tin	Mô tả	Cấu trúc (Header + Payload)
PUBLISH	Client (hoặc Broker) gửi thông điệp ứng dụng đến Topic.	Fixed Header (QoS, DUP, RETAIN) + Variable Header (Tên Topic, Packet Identifier) + Payload (Dữ liệu thực tế)
PUBACK	Phản hồi cho PUBLISH QoS 1, xác nhận đã nhận thông điệp.	Fixed Header + Variable Header (Packet Identifier, Mã lý do)
PUBREC, PUBREL, PUBCOMP	Các gói tin bắt tay (handshake) để đảm bảo giao tiếp QoS 2 (Exactly once):	
	- PUBREC (Received): Broker xác nhận đã nhận PUBLISH.	
	- PUBREL (Release): Client xác nhận đã sẵn sàng hoàn tất.	
	- PUBCOMP (Complete): Broker xác nhận đã hoàn tất giao dịch.	

d) MQTT Subscriber

- Luồng kết nối MQTT
- CONNECT → client gửi yêu cầu kết nối.
- CONNACK → broker xác nhận kết nối thành công

- SUBSCRIBE → client gửi yêu cầu đăng ký topic.
- SUBACK → broker xác nhận quyền đăng ký với QoS

SUBSCRIBE → client gửi yêu cầu đăng ký topic.

```
2025-10-01T17:03:37.123756 [MQTT] LISTENING
d2b265...hivemq.cloud:8883 topic=aquanova/devices/+/telemetry
2025-10-01T17:03:37.389492 [MQTT] CONNECTED with code=Success

2025-10-01T17:03:37.390013 [MQTT] SUBSCRIBE sent
topic=aquanova/devices/+/telemetry mid=1 result=0
```

Subscriber gửi gói CONNECT và broker phản hồi CONNACK với code Success, xác nhận kết nối MQTT

- topic: aquanova/devices/+/telemetry
- mid=1: ID duy nhất của gói đăng ký
- result=0: gửi thành công về phía client library

SUBACK → broker xác nhận quyền đăng ký với QoS

```
2025-10-01T17:03:37.601655 [MQTT] SUBACK received mid=1,
granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]
```

Thành phần	Subscribe topic	Ý nghĩa
Server/WebApp	aquanova/devices/+/telemetry	Nhận dữ liệu từ tất cả sensor
MCU Feeder	aquanova/devices/feeder/cmd	Nhận lệnh cho ăn
MCU Light	aquanova/devices/light/cmd	Nhận lệnh bật/tắt đèn
Mobile App	aquanova/devices/+/status, .../alert	Nhận trạng thái & cảnh báo
AI Engine	aquanova/devices/+/telemetry	Phân tích dữ liệu môi trường, đưa ra hành động

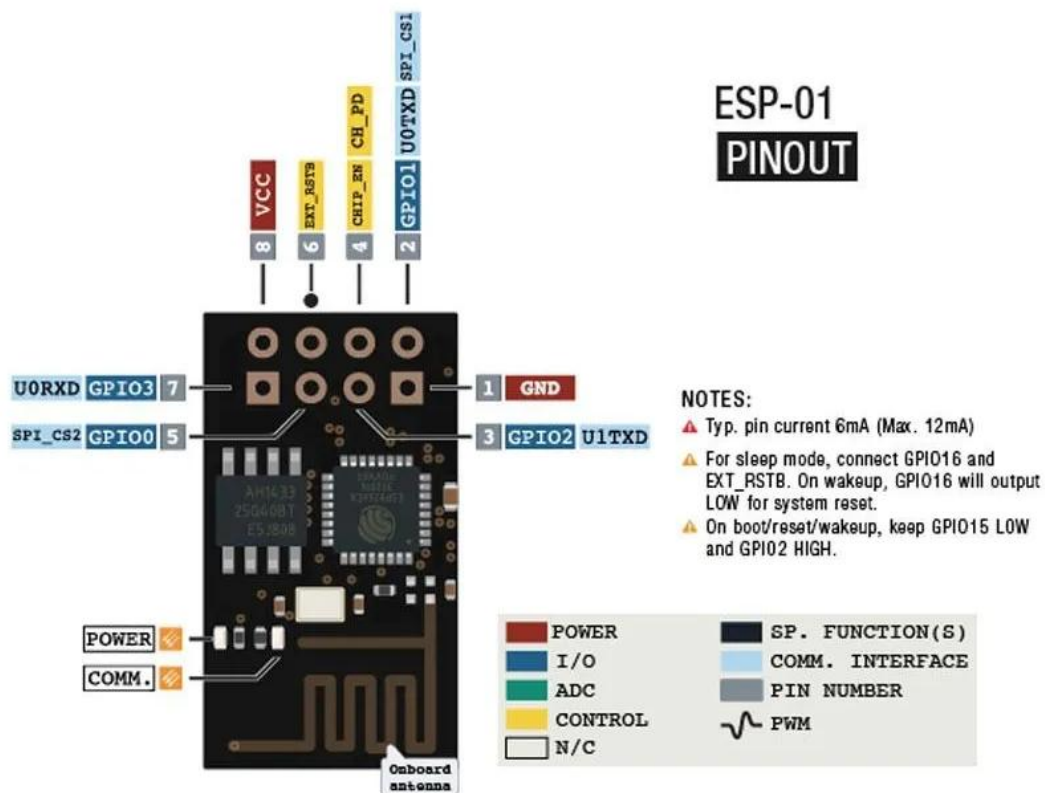
1.5. Thiết bị phần cứng

1.5.1. ESP8266 ESP-01

ESP8266 ESP-01 là một module IoT được thiết kế dựa trên vi điều khiển ESP8266 của công ty Espressif Systems. Đây là phiên bản module nhỏ gọn nhất trong các phiên bản của ESP8266 với kích thước chỉ 24mm x 14mm. ESP-01 có tích hợp sẵn một bộ phát WiFi, đủ để kết nối với mạng internet và truyền dữ liệu. Module này còn được tích hợp một cổng giao tiếp chuẩn UART, cho phép truyền dữ liệu giữa ESP8266 và các thiết bị khác.

Thông số kỹ thuật của Module Wifi ESP8266 ESP-01:

- Chip Wi-Fi: ESP8266EX
- Nguồn cấp: 3.0V ~ 3.6V DC
- Tiêu thụ dòng điện:
- Chế độ hoạt động: 80mA ~ 170mA
- Chế độ chờ: < 1.0mA
- Chuẩn giao tiếp: Wi-Fi 802.11 b/g/n
- Tốc độ truyền dữ liệu: 110 ~ 460800bps (tùy chọn)
- Điện áp: 3.3V DC
- Kích thước: 24mm x 14mm x 3mm
- Anten: PCB Anten hoặc IPEX anten ngoài (tùy chọn)



Hình 2: Sơ đồ chân ESP8266 ESP-01

1.5.2. Relay SRD-5VDC-SL-C

Module relay SRD-5VDC-SL-C hoạt động với điện áp đầu vào là 5V DC (Direct Current). Nó có khả năng chuyển đổi tải điện từ mạch điều khiển sang tải ngoại vi (ví dụ: đèn, motor, thiết bị điện...). Module này được điều khiển bằng nguồn điện DC và tín hiệu điều khiển từ mạch điện tử thông qua một đường tín hiệu kích hoạt (triggers) để mở hoặc đóng relay.



Hình 5: Relay SRD-5VDC-SL-C.

Bảng 3: Thông số kỹ thuật của Module relay SRD-5VDC-SL-C

Điện áp đầu vào	5V
Tiếp điểm	SPDT

Dòng điện chuyển mạch	10A
Kiểu chân	Xuyên lỗ
Series	SRD

1.5.3. SG90 Động Cơ Servo 180 Độ



Hình 6: SG90 Servo 180 độ

SG90 có kích thước nhỏ, giá thành thấp là loại động cơ servo phổ biến sử dụng nhiều nhất để làm các mô hình nhỏ hoặc các cơ cấu kéo không cần đến lực nặng, phù hợp với mọi người, đặc biệt là học sinh, sinh viên với nhu cầu nghiên cứu, ứng dụng, học tập,... Động cơ servo này có các bánh răng được làm bằng nhựa, được tích hợp sẵn Driver điều khiển động cơ, dễ dàng điều khiển góc quay bằng phương pháp điều độ rộng xung PWM.

- Phạm vi xung PWM 1ms - 2ms: góc quay tối đa 90 độ
- Phạm vi xung PWM 0.5ms - 2.5ms: góc quay tối đa 180 độ

Bảng 4: Thông số kỹ thuật của module SG90 Servo 180 Độ

Kích thước	21.5 x 11.8 x 22.7mm
Góc quay	180 độ
Trọng lượng	9g
Tốc độ không tải	0.12 giây / 60° (4.8V)
Mô-men xoắn	1.2-2.4 kg.cm (4.8V)
Nhiệt độ hoạt động	-30 → +60 ° C
Điện áp hoạt động	4.8 → 6 V dc

1.5.4. Cảm biến nhiệt độ MKE-S15 DS18B20



Hình 5: Cảm biến nhiệt độ MKE-S15 DS18B20

Cảm biến DS18B20 là một đầu dò nhiệt độ kỹ thuật số chống nước, được chế tạo với vỏ bọc kín nhằm đảm bảo độ bền và khả năng hoạt động trong nhiều điều kiện môi trường khắc nghiệt. Thiết bị được phát triển bởi Dallas Semiconductor (nay thuộc Maxim Integrated [9]) và được cung cấp dưới dạng dây dài tiêu chuẩn 1 mét, thuận tiện cho việc triển khai trong thực tế. DS18B20 hoạt động dựa trên nguyên lý chuyển đổi trực tiếp nhiệt độ sang tín hiệu số, nhờ bộ chuyển đổi tín hiệu tương tự – số (ADC) tích hợp với độ phân giải đến 12 bit. Cảm biến sử dụng giao thức truyền thông nối tiếp 1-Wire, cho phép mỗi thiết bị được định danh duy nhất bằng một mã nối tiếp 64 bit, đảm bảo khả năng kết nối và nhận diện nhiều cảm biến trên cùng một đường dữ liệu. Ngoài ra, DS18B20 có thể vận hành trong chế độ nguồn điện ký sinh, giúp giảm số chân kết nối cần thiết. Với đặc tính là cảm biến 1-Wire, DS18B20 chỉ yêu cầu một chân dữ liệu cùng chân GND để giao tiếp với vi điều khiển. Thiết bị hỗ trợ dải đo từ -55°C đến $+125^{\circ}\text{C}$, với độ chính xác khoảng $\pm 0.5^{\circ}\text{C}$ trong phạm vi thông thường và sai số lớn hơn ở biên nhiệt độ. Nguồn nuôi cho cảm biến dao động từ 3.0 V đến 5.5 V, với dòng tiêu thụ tối đa khoảng 1 mA. Một ưu điểm nổi bật của DS18B20 là chức năng cảnh báo nhiệt độ, cho phép người dùng thiết lập ngưỡng giới hạn cao hoặc thấp. Khi nhiệt độ vượt quá ngưỡng này, cảm biến sẽ phát tín hiệu cảnh báo qua đường dữ liệu.

Bảng 1.1: Bảng thông số và đặc điểm kỹ thuật cảm biến DS18B20

Mục	Chi tiết
Loại cảm biến	Cảm biến nhiệt độ kỹ thuật số có thể lập trình, giao tiếp qua giao thức 1-Wire

Điện áp hoạt động	3.0 V – 5.5 V
Độ chính xác	± 0.5 °C trong dải -10 °C đến +85 °C (<i>chính xác, không ± 5 °C</i>)
Dải nhiệt độ hoạt động	-55 °C đến +125 °C (tương đương -67 °F đến +257 °F)
Độ phân giải	Lựa chọn 9 đến 12 bit
Giao tiếp	1-Wire, chỉ cần một chân dữ liệu, không cần linh kiện ngoài
Mã định danh	Mỗi chip có ID 64-bit duy nhất, cho phép nhiều cảm biến dùng chung một bus dữ liệu
Tính năng đặc biệt	Hệ thống cảnh báo giới hạn nhiệt độ
Thời gian truy vấn	< 750 ms
Kết nối dây tiêu chuẩn	- Dây đỏ: VCC (3.0–5.5 V)- Dây đen: GND- Dây vàng: DATA
Thông số vật lý (vỏ bọc chống nước)	- Ống thép không gỉ, đường kính 6 mm, dài 35 mm- Cáp: đường kính 4 mm, dài 95 cm (~37.4 inch)- Đầu dò: đường kính 7 mm, dài 26 mm- Tổng chiều dài dây: ~6 feet
Bảo vệ	Vỏ thép không gỉ chống ẩm, chống nước, chống gỉ; bên trong có keo niêm phong và ống co nhiệt chống đoản mạch
Yêu cầu điện trở kéo lên	Bắt buộc có điện trở pull-up (4.7 k Ω) nối giữa dây Data và VCC để đảm bảo tín hiệu ổn định
Kết nối với vi điều khiển	- VCC ↔ nguồn 3.0–5.5 V- GND ↔ chân GND- Data ↔ chân digital I/O- Có điện trở kéo lên Data–VCC
Phần mềm hỗ trợ	Cần sử dụng thư viện 1-Wire chuyên dụng (ví dụ: OneWire.h, DallasTemperature.h trên Arduino)

1.5.5. Cảm biến đo độ đục MJKDZ Turbidity Sensor



Hình 6: Cảm biến đo độ đục MJKDZ Turbidity Sensor

Cảm biến độ đục MJKDZ hoạt động dựa trên nguyên lý tán xạ ánh sáng (Nephelometry). Trong cấu trúc của module, một nguồn sáng, thường là diode phát quang (LED), chiếu ánh sáng vào mẫu chất lỏng cần phân tích. Khi đi qua môi trường chứa các hạt lơ lửng, chùm sáng sẽ xảy ra hiện tượng tán xạ do sự tương tác với các hạt có kích thước, hình dạng và đặc tính quang học khác nhau. Mức độ tán xạ này phản ánh trực tiếp nồng độ các hạt trong dung dịch và do đó đặc trưng cho độ đục của mẫu. Một bộ dò quang (photodiode hoặc phototransistor) được bố trí ở góc thích hợp (thường là 90° so với hướng chiếu sáng) để thu nhận phần ánh sáng tán xạ. Cường độ ánh sáng mà bộ dò thu được tỷ lệ thuận với mật độ các hạt lơ lửng trong chất lỏng. Tín hiệu quang sau đó được chuyển đổi thành tín hiệu điện, trong đó biên độ tín hiệu điện tỷ lệ thuận với cường độ ánh sáng tán xạ, và do đó tỷ lệ thuận với độ đục của mẫu. Tín hiệu điện này được đưa vào mạch xử lý và vi điều khiển để thực hiện quá trình số hóa, hiệu chuẩn và tính toán giá trị độ đục theo các đơn vị chuẩn như NTU. Kết quả đo được hiển thị trên màn hình kỹ thuật số hoặc truyền tới hệ thống thu thập dữ liệu. Phương pháp này đảm bảo tính trực quan, nhanh chóng và chính xác, đồng thời phù hợp với các ứng dụng trong giám sát chất lượng nước sinh hoạt, nuôi trồng thủy sản.

Bảng 1.2: Bảng phân tích và thông số cảm biến MJKDZ Turbidity Sensor

Mục	Chi tiết
(1) Thông tin cảm biến – phân tích lựa chọn linh kiện	- Xuất xứ: Trung Quốc - Giá thành: Rẻ (~100–150k VND) - Ưu điểm: Dễ mua, chi phí thấp - Hạn chế: Độ chính xác và ổn định thấp hơn so với SEN0182
(2) Thông số kỹ thuật	- Điện áp hoạt động: 5 V - Ngõ ra: Analog (0 – 4.5 V), Digital (ngưỡng chỉnh bằng biến trở)

	- Dải đo: $\sim 0 - 1000$ NTU (độ chính xác kém hơn SEN0182) - Độ ổn định: Thấp, tín hiệu nhiễu nhiều, cần thêm mạch lọc
(3) Yêu cầu kết nối phần cứng	- Kết nối tương tự SEN0182 - Nguồn nuôi: 5 V - Ngõ ra AOUT $\rightarrow$ ADC của vi điều khiển (<i>cần chia áp khi dùng STM32 3.3 V</i>) - Khuyến nghị: Thêm tụ lọc ($0.1 - 1 \mu\text{F}$) chống nhiễu trên đường tín hiệu
(4) Giao thức phần mềm	- Đọc giá trị ADC và chuyển đổi thành điện áp - Hiệu chuẩn: Cần hiệu chuẩn thủ công bằng dung dịch chuẩn (<i>không có công thức chuyển đổi sẵn</i>) - Lọc tín hiệu: Nên lấy trung bình nhiều mẫu để giảm dao động và nhiễu

1.5.5. Cảm biến siêu âm HCSR04

Cảm biến siêu âm phát ra siêu âm, siêu âm sau khi chạm vào vật cản sẽ phản xạ lại và được thu bởi cảm biến. (**hiện tượng phản xạ sóng âm.**)

- Bộ phát (Trigger – Trig) của cảm biến phát ra một chùm sóng âm tần số 40 kHz trong không khí.
- Khi sóng âm gặp vật cản, nó sẽ phản xạ ngược trở lại và được bộ thu (Echo) của cảm biến ghi nhận.
- Thời gian từ lúc phát sóng đến khi nhận được sóng phản xạ được gọi là thời gian bay của sóng âm (*time of flight – ToF*).

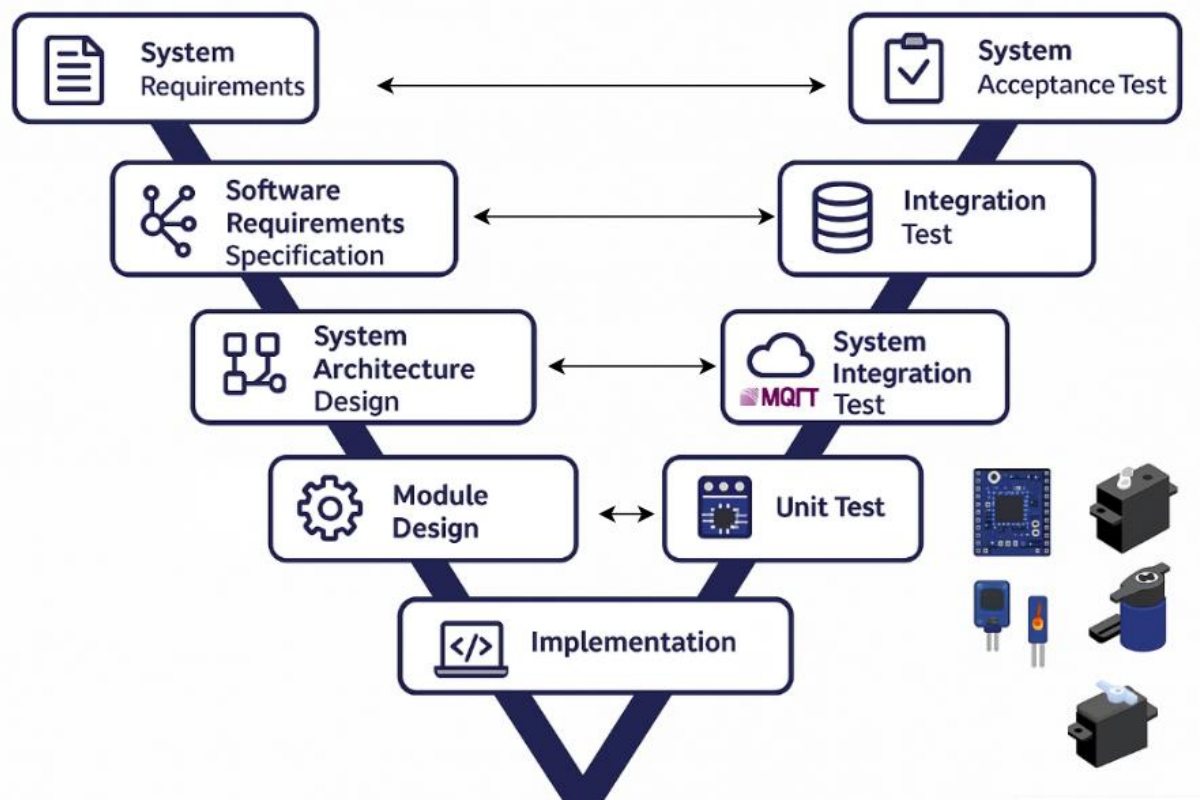
Do sóng âm truyền trong không khí với vận tốc $v \approx 343$ m/s (ở 25°C), ta có thể xác định khoảng cách đến vật cản dựa vào thời gian bay này. Vì vậy có thể đo khoảng cách bằng cách xác định thời gian từ lúc phát đến lúc nhận được xung phản xạ.

CHƯƠNG 2

NGHIÊN CỨU, THIẾT KẾ MÔ HÌNH

2.1. Quy trình các bước thực hiện

Mô hình chữ V (V model), hay mô hình xác minh (Verification) và xác nhận (Validation), là một mô hình vòng đời phát triển phần mềm (SDLC – Software Development Life Cycle). Mô hình ưu tiên việc kiểm thử (testing) sớm, một cách nghiêm ngặt trong suốt quá trình phát triển phần mềm và được biểu diễn trực quan như hình chữ V.[10]



Hình 7: Mô hình chữ V. Trong đó nhánh trái mô tả các giai đoạn thiết kế và đặc tả hệ thống (Verification) và Nhánh phải: mô tả các giai đoạn kiểm thử và đánh giá (Validation)

1. Xác định yêu cầu hệ thống (System Requirements)

Mục tiêu: Xác định nhu cầu tổng thể của người dùng và chức năng chính.

Đầu ra:

- Hệ thống tự động cho cá ăn theo lịch trình, điều khiển đèn
- Giám sát nhiệt độ, độ đục nước và lượng thức ăn theo thời gian thực.
- Điều khiển qua web (Flask backend + Firestore + MQTT).
- Cảnh báo khi vượt ngưỡng (qua MQTT và dashboard).

2. Kiểm thử tương ứng (System Acceptance Test)

Kiểm thử thực tế toàn hệ thống: lịch hoạt động đúng, sensor hoạt động, MQTT phản hồi.

3. Thiết kế yêu cầu phần mềm (Software Requirement Specification)

Mục tiêu: Phân rã chức năng thành module logic.

Đầu ra: Biểu đồ chức năng, use case, API endpoints.

Module đo nhiệt độ: DS18B20.

Module đo độ đục: cảm biến turbidity.

Module điều khiển servo feeder.

Flask backend: /feed, /schedule, /sensor.

Firestore: collection “Schedule”, “Telemetry”.

MQTT topics:

aquanova/telemetry

aquanova/control/feeder

aquanova/control/light

4. Kiểm thử tương ứng (Integration Test)

Kiểm tra giao tiếp giữa Flask ↔ MQTT ↔ STM32 hoạt động đúng.

5. Thiết kế kiến trúc hệ thống (System Architecture Design)

Mục tiêu: Xác định cấu trúc tổng thể phần cứng & phần mềm.

output: Sơ đồ khối hệ thống.

Hardware: STM32 + DS18B20 + Turbidity Sensor + Servo + WiFi module (ESP32).

Communication: MQTT (HiveMQ Cloud).

Software: Flask server + Firestore cloud + Web Dashboard.

6. Kiểm thử tương ứng (System Integration Test)

→ Kiểm tra kết nối MQTT, Flask REST API, database cập nhật realtime.

7. Thiết kế chi tiết module (Module Design)

Mục tiêu: Thiết kế từng module riêng biệt.

Module Sensor: đọc giá trị ADC, tính toán độ đục.

Module Feeder: điều khiển servo 0–180° với timer.

Module Network: publish/sub MQTT.

Flask Blueprint: telemetry.py, control.py, admin.py.

8. Kiểm thử tương ứng (Unit Test)

Test từng module độc lập: đọc cảm biến, servo quay đúng góc, MQTT gửi đúng payload.

9 Cài đặt (Implementation / Coding)

Mục tiêu: Lập trình và tích hợp code theo thiết kế.

MCU: STM32CubeIDE (C code).

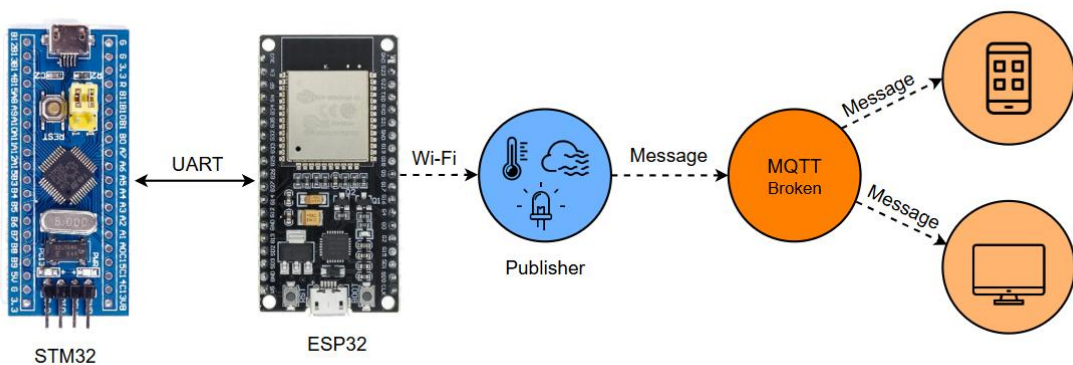
Server: Flask Python, MQTT client, Firestore (Firebase)

Frontend: HTML/JS giao diện điều khiển.

9. Kiểm thử tương ứng (Code Verification / Unit Testing)

Chạy test, debug từng phần bằng console log hoặc serial monitor.

2.2. Mô hình tổng quan hệ thống



Hình 8: Sơ đồ mô hình tổng quan sử dụng MQTT vào ESP8266 kết hợp STM32

Để xây dựng một hệ thống IoT hiệu quả, việc thiết kế kiến trúc hệ thống là rất quan trọng. Một trong những mô hình kiến trúc phổ biến là kiến trúc 3 lớp [3], bao gồm:

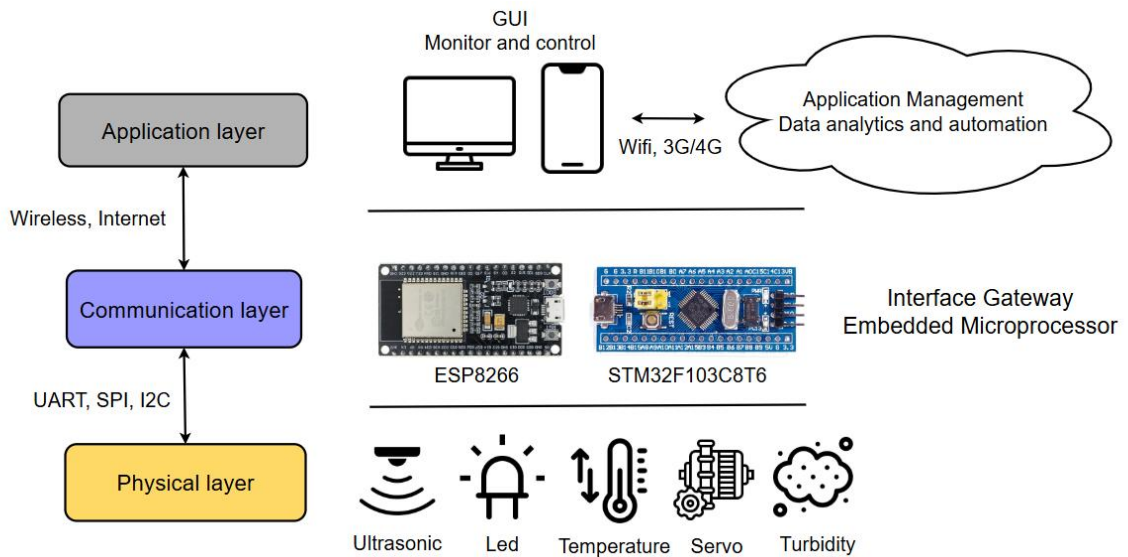
- Lớp cảm nhận (Physical Layer): Chịu trách nhiệm thu thập dữ liệu từ môi trường thông qua các cảm biến và thiết bị đầu vào.
- Lớp mạng (Communication Layer): Truyền tải dữ liệu thu thập được từ lớp cảm nhận lên lớp xử lý hoặc lưu trữ. Xử lý và phân tích dữ liệu để đưa ra các quyết định hoặc hành động phù hợp.
- Lớp ứng dụng (Application Layer): Cung cấp giao diện người dùng và các dịch vụ cụ thể dựa trên kết quả xử lý từ các lớp trước.

Theo nghiên cứu của Burhan et al. (2018), kiến trúc 3 lớp này giúp phân tách rõ ràng các chức năng trong hệ thống, từ việc thu thập dữ liệu đến cung cấp dịch vụ cho người dùng cuối, từ đó nâng cao hiệu quả và khả năng mở rộng của hệ thống IoT. Dựa vào đó, nhóm đề xuất hệ thống trình bày theo mô hình kiến trúc 3 lớp.

Lớp đầu tiên là lớp vật lý (Physical Layer), đóng vai trò là giao diện giữa hệ thống số và thế giới vật chất. Lớp này tương đương trực tiếp với Lớp 1 (Physical Layer) của Mô hình OSI. Việc lựa chọn lớp này là nhằm cô lập trách nhiệm thu thập dữ liệu thô (Data Acquisition) từ môi trường thông qua các cảm biến (như Nhiệt độ, Độ ẩm, Siêu âm) và thực hiện hành động (Actuation) thông qua các thiết bị chấp hành (như Led, Servo). Bằng cách này, mọi vấn đề liên quan đến tín hiệu điện, cơ học, và chuyển đổi từ tín hiệu vật lý sang tín hiệu số cơ bản đều được xử lý tại đây, đảm bảo tính mô-đun và dễ dàng thay thế hoặc mở rộng các thiết bị đầu cuối.

Lớp truyền thông (Communication Layer) là trung tâm của hệ thống, thực hiện chức năng như một Cổng giao tiếp (Gateway). Lớp này bao gồm các bộ vi xử lý nhúng mạnh mẽ như ESP8266 và STM32F103C8T6, tương ứng với chức năng của Lớp 2 (Data Link Layer) và Lớp 3 (Network Layer) của OSI. Lớp này có hai nhiệm vụ cốt lõi là tiền xử lý cục bộ (Local Processing) dữ liệu thô nhận được từ Lớp Vật lý thông qua các giao thức bus tốc độ thấp (UART, SPI, I2C); thứ hai là thiết lập và quản lý kết nối mạng (Network Connectivity) qua WiFi/3G/4G để truyền dữ liệu đã được xử lý một cách đáng tin cậy lên lớp ứng dụng hoặc nhận lệnh điều khiển xuống. Sự phân tách này tối ưu hóa việc sử dụng tài nguyên và đảm bảo giao tiếp liền mạch giữa các lớp.

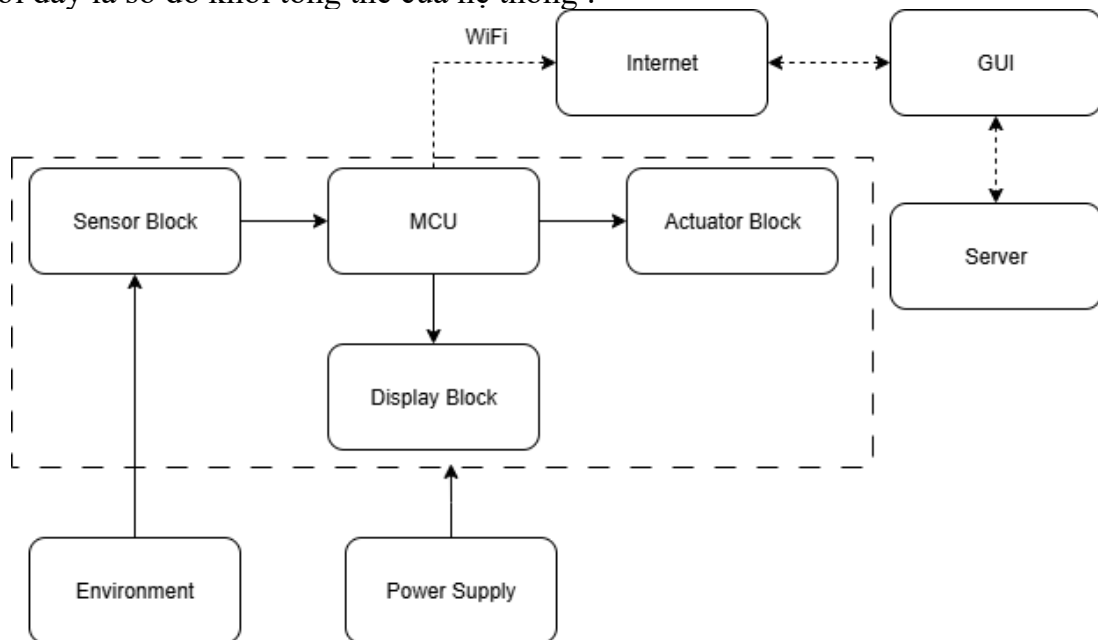
Lớp ứng dụng (Application Layer) là lớp cao nhất, tập trung hoàn toàn vào nghiệp vụ hệ thống và trải nghiệm người dùng, tương ứng với Lớp 7 (Application Layer) của OSI. Lớp này bao gồm các dịch vụ như Giao diện người dùng (GUI), Giám sát và Điều khiển, cùng với các tính năng nâng cao như Phân tích dữ liệu và Tự động hóa (thường chạy trên Cloud). Lý do lựa chọn lớp này là để tách biệt hoàn toàn logic kinh doanh khỏi phần cứng, cho phép hệ thống biến dữ liệu thô thành thông tin có giá trị (Meaningful Information), hiển thị trực quan và hỗ trợ ra quyết định. Việc sử dụng tài nguyên máy chủ/Cloud tại lớp này cũng giải phóng các thiết bị nhúng (Lớp Truyền thông) khỏi các tác vụ tính toán nặng, mang lại sự linh hoạt, khả năng mở rộng và cập nhật tính năng dễ dàng cho toàn bộ hệ thống.



Hình 9: Kiến trúc tổng quan của hệ thống iot 3 lớp cho việc giám sát và cung cấp thức ăn tự động cho cá.

2.2. Sơ đồ khối hệ thống

Dưới đây là sơ đồ khối tổng thể của hệ thống :



Hình 10: Sơ đồ hệ thống

Hệ thống được xây dựng bao gồm có 4 khối kết nối với nhau:

- Khối cảm biến: khối này dùng cảm biến đo siêu âm, servo, camera nhằm thực hiện việc theo dõi quan sát môi trường sống của cá và cảnh báo khi có vấn đề xảy ra
- (MCU) Microcontroller Unit: đây là khối trang bị các thuật toán, hàm để tương tác với dữ liệu được thu lại từ khối cảm biến và đồng thời chuyển dữ liệu đã xử lý đến khối điều khiển và khối hiển thị.

- Khối cơ cấu chấp hành: khối này bao gồm các relay, nhận các tín hiệu dùng cho mục đích điều khiển từ khối xử lý trung tâm để đóng cắt các thiết bị điện bên ngoài.
- Khối hiển thị: là một màn hình LCD sử dụng để đưa ra các thông báo cho người dùng biết các thông tin đã và đang được xử lý trong khối xử lý trung tâm.
- Khối nguồn: là khối cấp nguồn 5VDC cho các khối cảm biến, khối xử lý trung tâm và khối điều khiển.
- Database: Hệ thống sử dụng các database để tiện cho giao tiếp giữa cần cứng với Web.
- Web: Là một giao diện cơ bản được xây dựng từ HTML, CSS và JS, nhằm mục đích thu thập, xử lý và trực quan hóa các thông tin liên quan đến môi trường sống hồ cá như độ đục, nhiệt độ, lượng thức ăn.

Dựa trên yêu cầu đặt ra của sơ đồ tổng thể của hệ thống và các yêu cầu về mặt tính năng cũng như kỹ thuật, nhóm đã tìm hiểu, nghiên cứu và chọn ra các phần cứng phù hợp với từng khối, đáp ứng được các yêu cầu kể trên.

2.3. Sơ đồ chi tiết khối chính

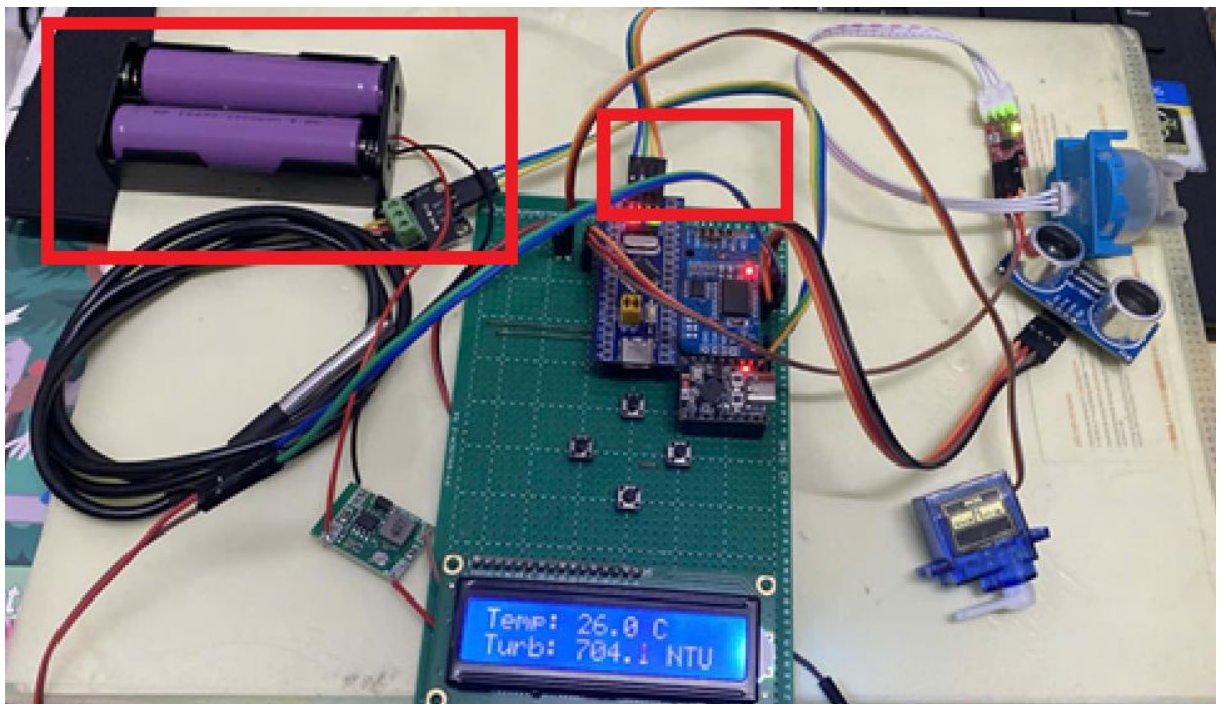
2.3.1. Khối nguồn (Bình)

Trong bất kỳ hệ thống nhúng nào, khối nguồn (Power Supply Unit - PSU) đóng vai trò là xương sống, quyết định trực tiếp đến sự ổn định, chính xác và tuổi thọ của toàn bộ mạch. Một hệ thống có vi điều khiển (ViDK), cơ cấu chấp hành công suất lớn (như động cơ servo) và các tải phụ (như đèn) đặt ra một bài toán thiết kế nguồn phức tạp. Việc cấp chung một nguồn duy nhất cho tất cả các thành phần thường dẫn đến hiện tượng sụt áp, nhiễu điện và gây ra sự hoạt động sai lệch, đặc biệt là với các ViDK nhạy cảm như STM32. Để đảm bảo sự ổn định cho toàn hệ thống, việc phân chia nguồn từ pin thành các nhánh riêng biệt cho từng loại tải là giải pháp tối ưu. Mô hình được đề xuất bao gồm hai nhánh chính:

- Nguồn 1: Cấp cho khối điều khiển (STM32) và khối công suất động cơ (Servo).
 - Nguồn cấp: Sử dụng hai cell pin Lithium mắc nối tiếp, cung cấp điện áp danh định khoảng 7.4V (đầy tải là 8.4V).
 - Mạch chuyển đổi: Điện áp từ pin được đưa qua một mạch hạ áp xung (Buck Converter) để tạo ra một mức điện áp ổn định là 5V.
 - Phân phối tải: Vi điều khiển STM32 dùng nguồn 5V này được cấp cho bo mạch chứa STM32. Thông thường, trên các bo mạch phát triển, nguồn 5V này sẽ tiếp tục đi qua một bộ ổn áp tuyến tính nhiễu thấp (LDO) tích hợp sẵn để tạo ra điện

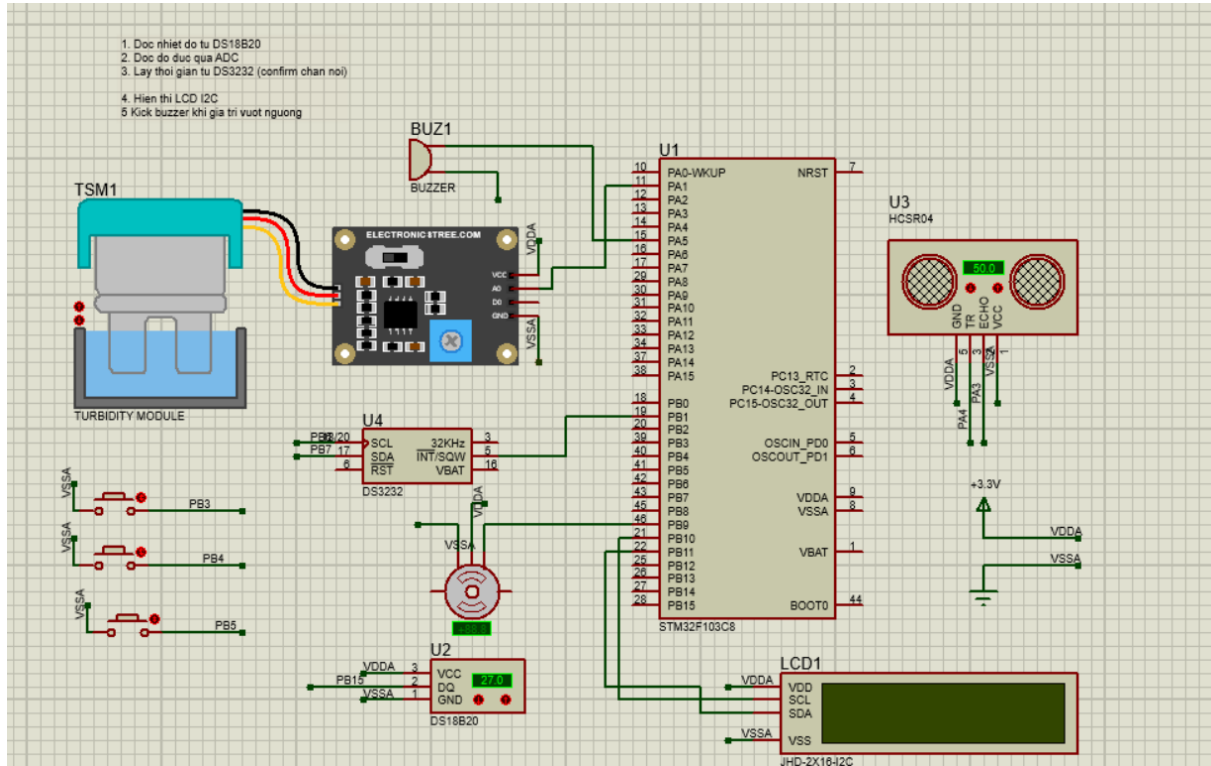
áp 3.3V siêu sạch, cấp trực tiếp cho nhân VĐK. Động cơ Servo cũng dùng nguồn 5V từ mạch hạ áp cũng được cấp song song cho động cơ servo. Một tụ hóa có điện dung lớn được mắc song song ngay tại chân nguồn của servo.

- Nguồn 2: Cấp riêng cho đèn thông qua relay.
 - Nguồn cấp: Sử dụng một nguồn pin Lithium riêng biệt (có thể là một hoặc nhiều cell tùy thuộc vào yêu cầu điện áp của đèn).
 - Cơ cấu đóng ngắt: Đèn được kết nối với nguồn pin này thông qua một relay. Cuộn hút của relay được điều khiển bởi tín hiệu từ vi điều khiển STM32, nhưng nguồn cấp cho tiếp điểm công suất của relay hoàn toàn độc lập.



Hình 11: Mô hình được cấp nguồn từ bộ pin Li-ion 3.7 V (2 cell, nối tiếp qua module sạc – chuyển áp), cung cấp điện cho vi điều khiển (ESP32/STM32) và các cảm biến thông qua mạch ổn áp 5 V – 3.3 V.

2.3.2. Khối cảm biến



Hình 12: Sơ đồ chân các cảm biến trong mô hình

Khối cảm biến gồm các cảm biến chính: DS18B20 đo nhiệt độ nước, Turbidity sensor đo độ đục, và HC-SR04 đo mức nước. Các tín hiệu thu được được đưa về vi điều khiển STM32F103C8 để xử lý. Ngoài ra, mô hình còn tích hợp DS3232 RTC để định thời và LCD I2C hiển thị thông số. Khi giá trị đo vượt ngưỡng, buzzer sẽ phát tín hiệu cảnh báo, giúp người dùng theo dõi tình trạng nước trong hồ cá một cách trực quan.

2.3.3. Khối xử lý trung tâm

Trong hệ thống, khối trung tâm bao gồm hai vi điều khiển chính là STM32 và ESP32. STM32 đảm nhiệm vai trò điều khiển và thu thập dữ liệu cảm biến, còn ESP32 đóng vai trò xử lý truyền thông mạng WiFi và gửi dữ liệu lên hệ thống IoT. Để hai vi điều khiển có thể trao đổi dữ liệu với nhau một cách hiệu quả, chuẩn giao tiếp UART (Universal Asynchronous Receiver – Transmitter) được lựa chọn làm phương thức kết nối chính. Mục đích của việc kết nối UART là để thiết lập một kênh truyền dữ liệu nối tiếp song công giữa STM32 và ESP32. UART là một trong những giao thức có kết nối đơn giản và hiệu quả nhất. Việc kết nối giữa STM32 và ESP32 chỉ cần hai dây cơ bản gồm: TX (Transmit) của STM32 nối với RX (Receive) của ESP32, RX của STM32 nối với TX của ESP32 chung để đảm bảo tín hiệu truyền ổn định. So với các giao thức khác

như SPI (cần ít nhất 4 dây) hay I2C (đòi hỏi cấu hình phức tạp hơn), UART giúp tiết kiệm chân kết nối, đơn giản hóa mạch phần cứng và thuận tiện trong thiết kế hệ thống nhúng.

UART được hỗ trợ trực tiếp bởi phần cứng của cả hai vi điều khiển, do đó việc truyền và nhận dữ liệu diễn ra gần như tự động, không tiêu tốn nhiều tài nguyên xử lý. Các hàm lập trình như HAL_UART_Transmit() và HAL_UART_Receive() trong thư viện HAL của STM32 hay Serial.read() và Serial.write() trong ESP32 (khi sử dụng Arduino framework) giúp việc lập trình trở nên dễ dàng, nhanh chóng. Người thiết kế chỉ cần gọi một hàm đơn giản là có thể gửi hoặc nhận dữ liệu giữa hai thiết bị. Một ưu điểm quan trọng khác của UART là khả năng giao tiếp song công (Full-Duplex), nghĩa là STM32 có thể gửi dữ liệu cảm biến sang ESP32 trong khi ESP32 vẫn có thể đồng thời gửi lại các lệnh điều khiển như “bật đèn”, “cho ăn” hoặc “thay đổi chế độ hẹn giờ”. Cơ chế này giúp hệ thống hoạt động trơn tru và phản hồi nhanh chóng, rất phù hợp với các ứng dụng IoT yêu cầu thời gian thực. UART còn rất phổ biến và dễ dàng trong việc kiểm tra, gỡ lỗi (debug). Chỉ cần một mạch chuyển USB-to-UART giá rẻ, người dùng có thể kết nối hệ thống với máy tính và theo dõi trực tiếp dữ liệu đang được truyền qua lại giữa STM32 và ESP32 trên các phần mềm như PuTTY hoặc Tera Term. Nhờ vậy, việc phát hiện và khắc phục lỗi trong quá trình phát triển trở nên dễ dàng và nhanh chóng hơn.

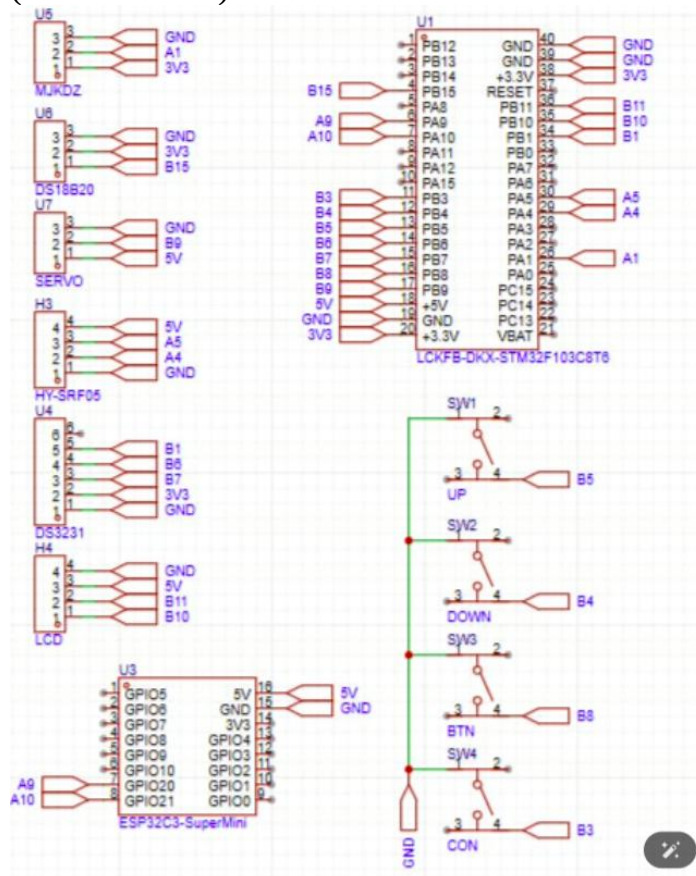
2.4. Quy trình các bước thực hiện

2.4.1. Xác định yêu cầu và linh kiện

Chức năng hệ thống	Mô tả yêu cầu	Linh kiện sử dụng
Đo nhiệt độ nước	Cảm biến chính xác, chống nước, sai số nhỏ $\pm 0.5^{\circ}\text{C}$	Cảm biến DS18B20
Đo độ đục của nước	Đo độ trong/đục để đánh giá chất lượng nước	Turbidity sensor (TSM1 module)
Đo mức nước hồ	Xác định mực nước để cảnh báo thiếu nước	HC-SR04 Ultrasonic sensor
Xử lý và hiển thị dữ liệu	Đọc giá trị cảm biến, hiển thị LCD, cảnh báo khi vượt ngưỡng	STM32F103C8, LCD I2C 16×2, Buzzer
Kết nối Internet và truyền dữ liệu	Gửi dữ liệu cảm biến lên server qua MQTT, nhận lệnh điều khiển	ESP32-C3 Mini, Wi-Fi module tích hợp
Đồng bộ thời gian	Lưu và hiển thị giờ thực, hỗ trợ hẹn lịch cho ăn	RTC DS3232 module
Điều khiển cơ cấu cho ăn	Quay servo định lượng thức ăn theo lịch hoặc lệnh tay	Servo motor SG90
Cấp nguồn cho hệ thống	Cung cấp điện ổn định 5 V–3.3 V, có thể dùng pin hoặc adapter	Nguồn pin Li-ion + Module sạc/ổn áp

2.4.2. Sơ đồ nguyên lý

(to be continue)

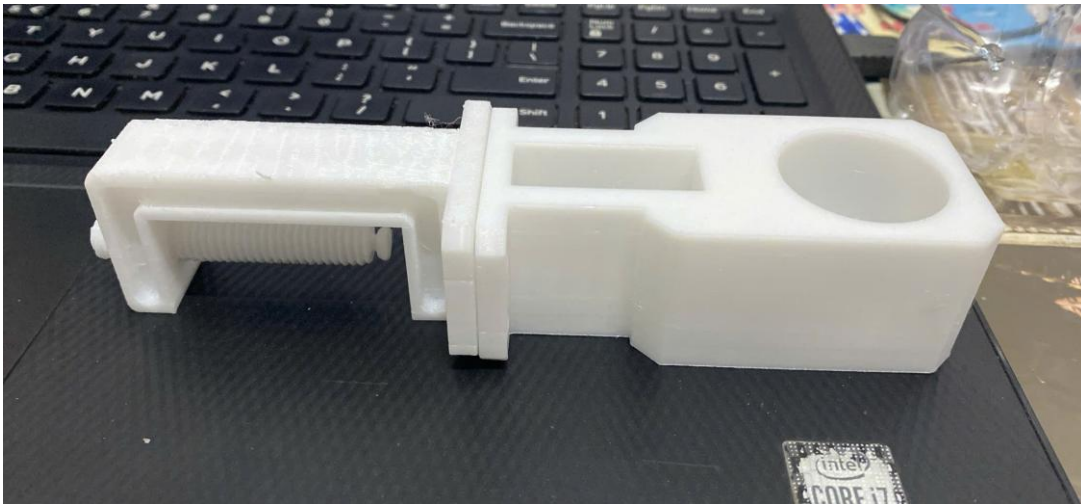


2.4.3. Thiết kế cơ khí

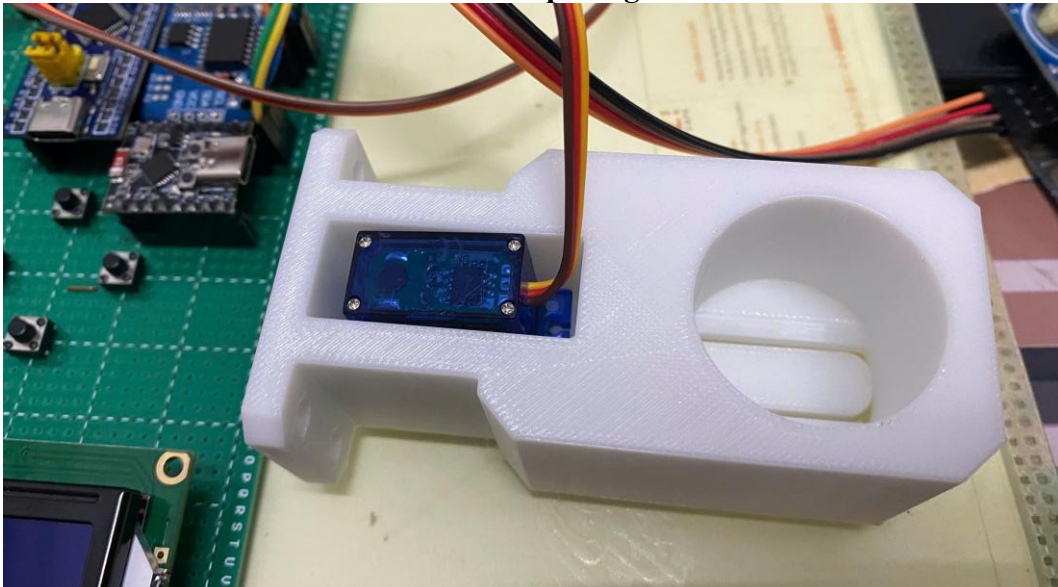
Mô hình cơ khí được xây dựng dựa trên nguyên tắc tối giản hóa, tận dụng triệt để trọng lực để cấp liệu và sử dụng một cơ cấu chấp hành trực tiếp để kiểm soát dòng chảy. Thiết kế này tập trung vào độ tin cậy, chi phí thấp và sự đơn giản trong chế tạo cũng như vận hành, loại bỏ các chi tiết truyền động phức tạp như trục xoắn hay băng tải. Cấu trúc của thiết bị bao gồm một chai nhựa thông thường, được đảo ngược và cố định chắc chắn trên một giá đỡ, đóng vai trò như một bể chứa (hopper) tiện lợi. Thiết kế này cho phép thức ăn dạng hạt tự động dồn xuống miệng chai dưới tác dụng của trọng lực, luôn ở trạng thái sẵn sàng để được phân phối. Điểm cốt lõi của cơ cấu chấp hành nằm ở một động cơ servo được định vị chính xác ngay tại miệng ra của chai. Cánh tay của servo (servo horn), hoặc một tấm gạt nhỏ được gắn vào đó, hoạt động như một van chặn trực tiếp, đóng vai trò như một cánh cửa đóng-mở linh hoạt.

Nguyên lý hoạt động của thiết bị diễn ra một cách tuần tự và rõ ràng. Ở trạng thái chờ, servo giữ cánh tay gạt ở vị trí đóng kín miệng chai, ngăn không cho thức ăn rơi ra ngoài. Khi nhận được tín hiệu điều khiển từ vi điều khiển theo lịch trình đã được cài đặt,

servo sẽ lập tức quay, di chuyển tấm chặn ra khỏi miệng chai. Hành động này tạo ra một khoảng hở, cho phép thức ăn rơi tự do xuống bể cá. Lượng thức ăn được định lượng gián tiếp thông qua khoảng thời gian mà van được giữ ở trạng thái mở; thời gian mở càng lâu, lượng thức ăn rơi xuống càng nhiều. Sau khi hết thời gian quy định, servo sẽ quay trở về vị trí ban đầu, đóng kín cửa ra và kết thúc một chu trình cấp liệu. Đây là một giải pháp cơ khí thông minh, tận dụng các linh kiện phổ biến để tạo ra một hệ thống tự động hóa hiệu quả. Mặc dù độ chính xác về định lượng không cao bằng các cơ cấu phức tạp hơn như trục xoắn, thiết kế này hoàn toàn đáp ứng được yêu cầu của các ứng dụng phổ thông, đặc biệt là khi ưu tiên sự đơn giản, độ bền và tối ưu hóa chi phí



Hình 13: Giá đỡ phân giữ thức ăn và servo



Hình 14: Cơ khí cho ăn đã gắn servo và que chặn

2.4.4. Thiết kế nhúng

Thiết kế nhúng tập trung vào việc lập trình cho vi điều khiển STM32F103C8, đóng vai trò là bộ xử lý trung tâm của hệ thống. STM32 nhận dữ liệu từ các cảm biến (DS18B20

đo nhiệt độ, cảm biến độ đục đo chất lượng nước, HC-SR04 đo mực nước) và xử lý để hiển thị thông số lên màn hình LCD I2C. Đồng thời, hệ thống được lập trình để cảnh báo khi giá trị vượt ngưỡng và điều khiển cơ cấu cho ăn tự động theo tín hiệu hẹn giờ từ DS3231 RTC hoặc lệnh thủ công của người dùng.



Hình 15: Sơ đồ Use Case các tính năng của mô hình, thể hiện mối quan hệ giữa người dùng và các chức năng chính mà hệ thống thực hiện. Người dùng có thể xem thông tin giám sát (nhiệt độ, độ đục, mức thức ăn, thời gian), đặt lịch hoặc kích hoạt cho ăn ngay

Lưu đồ giải thuật toàn hệ thống: to be continue

2.4.4.1. STM32

a) Module độ đục

Module MJKDZ đo độ đục là một loại cảm biến quang học được sử dụng để xác định mức độ trong hay đục của chất lỏng, đặc biệt là nước. Thiết bị này thường được ứng dụng trong các hệ thống giám sát chất lượng nước, xử lý nước thải, hồ nuôi thủy sản hoặc các dự án IoT môi trường. Nguyên lý làm việc của module dựa trên hiện tượng tán xạ ánh sáng khi có các hạt lơ lửng trong nước. Về cấu tạo, module MJKDZ gồm hai phần chính là đầu dò cảm biến và mạch xử lý tín hiệu.

Đầu dò cảm biến là phần tiếp xúc trực tiếp với môi trường nước, chứa một đèn LED hồng ngoại dùng để phát sáng và một photodiode để thu ánh sáng phản xạ hoặc truyền qua chất lỏng. Khi nguồn sáng phát ra đi qua nước, lượng ánh sáng đến được photodiode sẽ phụ thuộc vào mức độ trong suốt của nước. Nếu nước trong, ánh sáng sẽ truyền qua dễ dàng, cường độ ánh sáng thu được lớn, tín hiệu điện tạo ra sẽ cao. Ngược lại, nếu nước đục do chứa nhiều tạp chất lơ lửng, các hạt này sẽ làm tán xạ và hấp thụ ánh sáng, khiến lượng ánh sáng đến photodiode giảm đi, dẫn đến điện áp đầu ra thấp hơn.

Mạch xử lý tín hiệu nằm trên bo mạch MJKDZ có nhiệm vụ khuếch đại và so sánh tín hiệu điện từ cảm biến quang học. Trên mạch thường có một IC so sánh LM393, một

biến trở điều chỉnh ngưỡng, cùng với một số điện trở và tụ điện dùng để ổn định và lọc nhiễu tín hiệu. Mạch cung cấp hai ngõ ra:

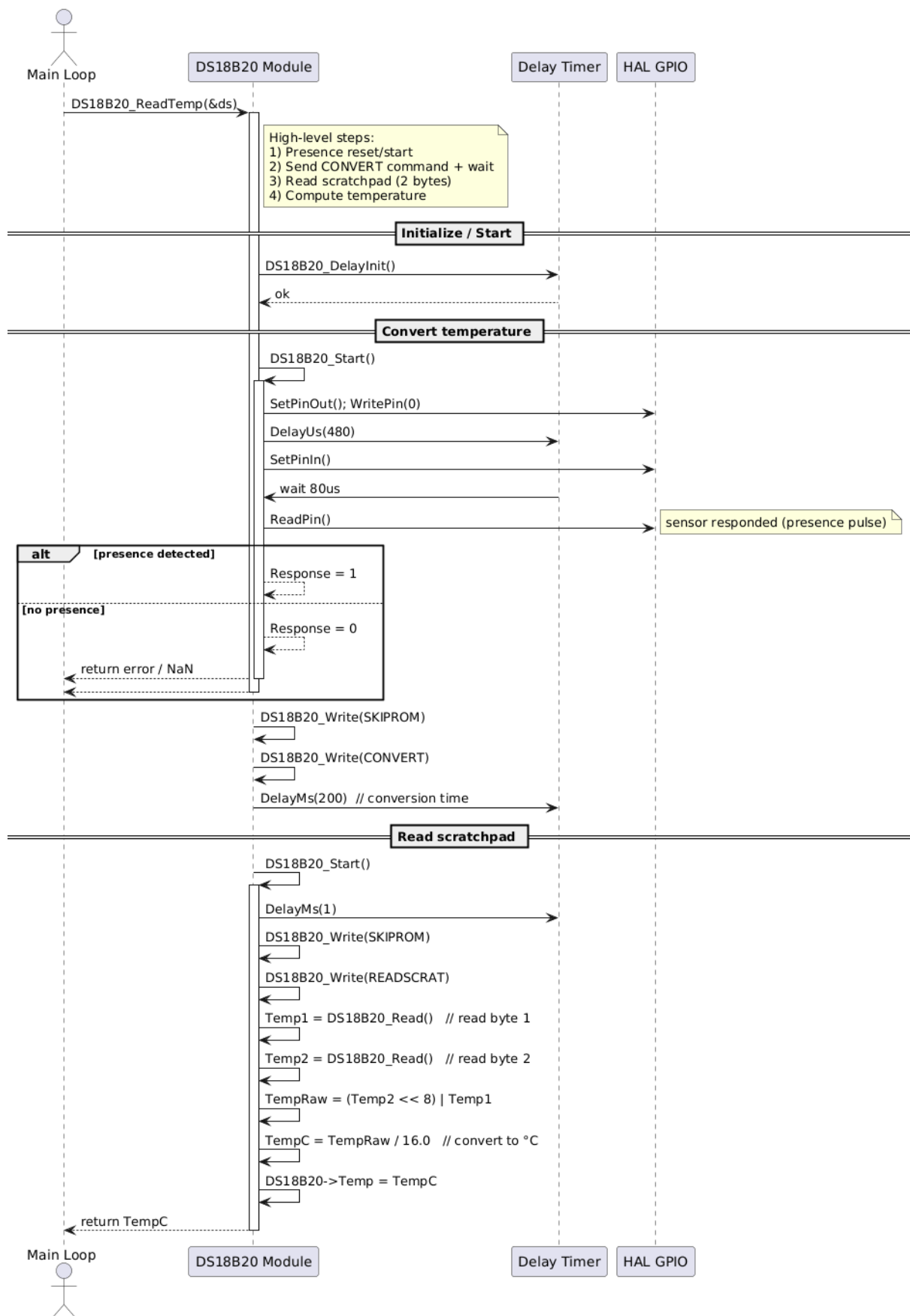
- Ngõ ra tương tự (AO – Analog Output), tạo ra điện áp tỉ lệ tuyến tính với độ đục của nước. Ngõ ra này thường được đưa vào bộ chuyển đổi ADC của vi điều khiển để đọc giá trị và xử lý định lượng.
- Ngõ ra số (DO – Digital Output), được tạo ra nhờ mạch so sánh bên trong LM393. Khi độ đục vượt qua ngưỡng được thiết lập bằng biến trở, đầu ra DO sẽ thay đổi trạng thái logic (thường là từ mức cao sang mức thấp).

Module hoạt động với điện áp nguồn 5V DC (một số phiên bản có thể dùng được 3.3V), và có bốn chân kết nối chính: VCC, GND, AO và DO. Trong quá trình đo, người dùng có thể điều chỉnh biến trở để đặt mức ngưỡng phù hợp với yêu cầu ứng dụng thực tế.

Cấu trúc thư viện và nguyên lý hoạt động

Tầng	Chức năng chính	Hàm tiêu biểu
Low Level (Phần cứng)	Điều khiển chân GPIO, tạo xung trễ micro-giây	DS18B20_SetPinOut(), DS18B20_SetPinIn(), DS18B20_DelayUs()
Middle Level (Giao thức 1-Wire)	Truyền nhận dữ liệu theo chuẩn 1-Wire	DS18B20_Start(), DS18B20_Write(), DS18B20_Read()
High Level (Ứng dụng)	Giao diện chính cho người dùng lập trình	DS18B20_Init(), DS18B20_ReadTemp()

b) Module nhiệt độ



Hình 16: Sơ đồ tuần tự cho module cảm biến nhiệt độ DS18B20

Sơ đồ tuần tự mô tả quá trình đo và đọc nhiệt độ từ cảm biến DS18B20 trong chương trình. Khi hàm DS18B20_ReadTemp(&ds) được gọi từ Main Loop, mô-đun DS18B20 tiến hành các bước chính gồm: khởi tạo, chuyển đổi nhiệt độ, đọc dữ liệu và tính toán kết quả.

Khi hàm DS18B20_ReadTemp() được kích hoạt từ Main Loop, mô-đun DS18B20 trước tiên thực hiện giai đoạn khởi tạo. Trong giai đoạn này, hàm DS18B20_DelayInit() được gọi để chuẩn bị bộ định thời, đảm bảo các khoảng trễ cần thiết được thiết lập chính xác. Sau đó, vi điều khiển gửi tín hiệu reset thông qua SetPinOut() và WritePin(0) nhằm yêu cầu cảm biến phản hồi tín hiệu. Cảm biến DS18B20 sẽ phản hồi bằng cách kéo mức điện áp xuống trong một khoảng thời gian ngắn. Việc có hoặc không có phản hồi này giúp hệ thống xác định xem cảm biến đang hoạt động bình thường hay không. Nếu phát hiện được tín hiệu hiện diện, biến phản hồi Response được gán bằng 1 và ngược lại, nếu không có tín hiệu, hàm sẽ trả về giá trị lỗi (NaN) và kết thúc quá trình đọc.

Tiếp theo, hệ thống gửi lệnh SKIP ROM và CONVERT T để yêu cầu cảm biến thực hiện đo nhiệt độ. Sau thời gian chờ khoảng 200 ms cho quá trình chuyển đổi, mô-đun gửi tiếp các lệnh SKIP ROM và READ SCRATCHPAD để đọc hai byte dữ liệu nhiệt độ lưu trong bộ nhớ tạm của cảm biến. Hai byte này được ghép lại thành giá trị thô TempRaw, sau đó chuyển đổi sang độ C bằng công thức:

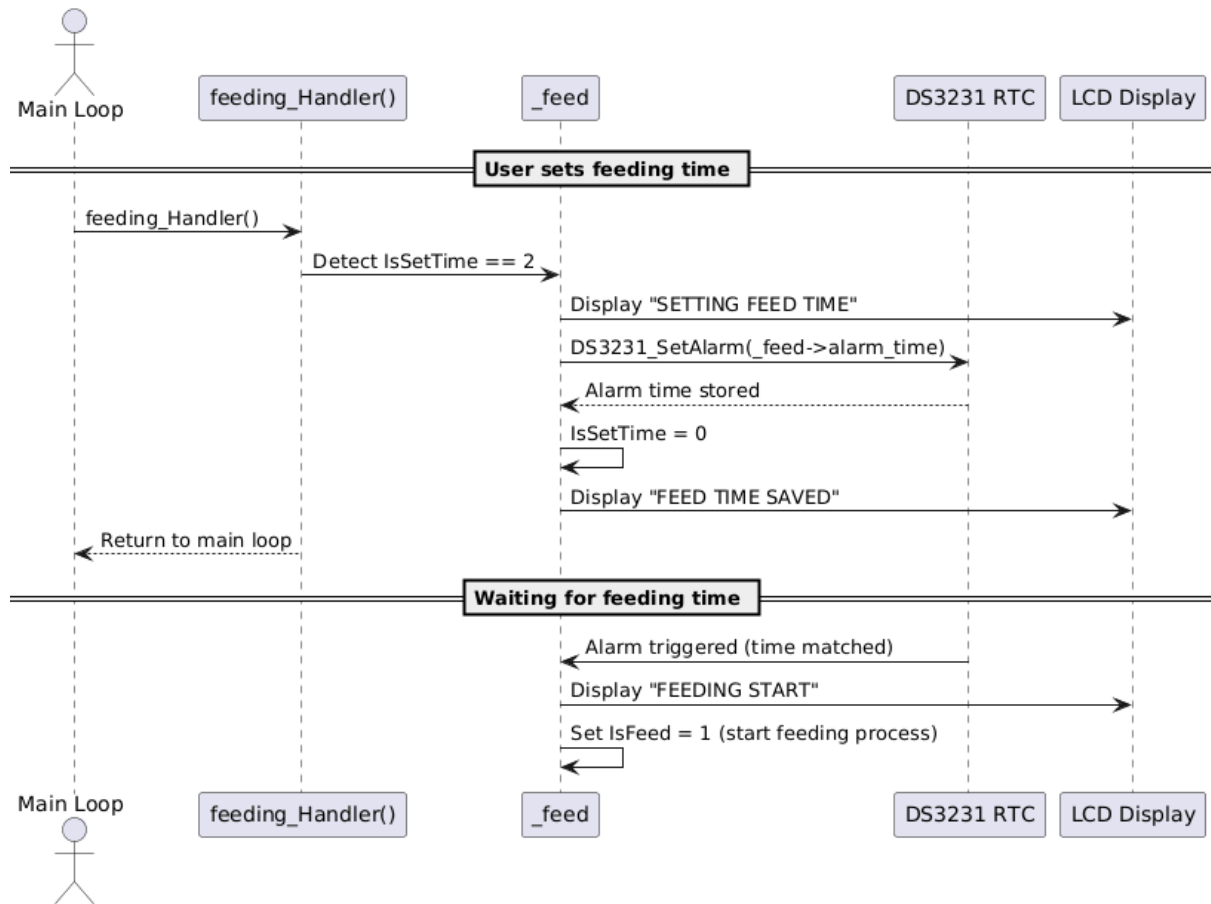
$$TempC = TempRaw / 16.0$$

Giá trị kết quả được lưu vào biến DS18B20->Temp và trả về cho Main Loop để hiển thị hoặc xử lý tiếp theo.

Thư viện đo nhiệt độ

Tầng	Hàm tiêu biểu	Chức năng chính
Low Level (Hardware)	DS18B20_SetPinOut/In(), DS18B20_ReadPin(), DS18B20_DelayUs()	Điều khiển trực tiếp chân GPIO, tạo xung delay chính xác ở cấp vi giây
Middle Level (Protocol)	DS18B20_Start(), DS18B20_Write(), DS18B20_Read()	Thực thi giao thức 1-Wire (khởi tạo, đọc/ghi bit, nhận phản hồi presence)
High Level (Application)	DS18B20_Init(), DS18B20_ReadTemp()	Hàm ứng dụng cao nhất – khởi tạo cảm biến và đọc giá trị nhiệt độ thực tế

c) Module đặt lịch cung cấp thức ăn



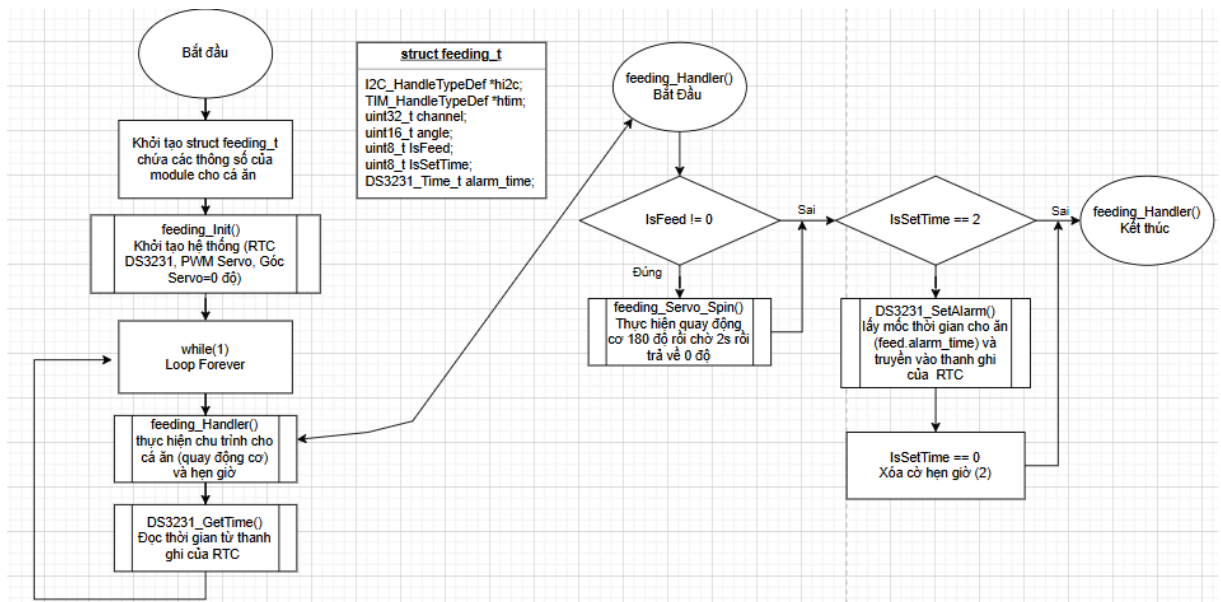
Hình 17: Sơ đồ tuần tự cho module đặt lịch cung cấp thức ăn

Mô-đun đặt lịch cung cấp thức ăn có nhiệm vụ thiết lập và quản lý thời gian cho ăn tự động thông qua tương tác giữa vi điều khiển, người dùng và mô-đun đồng hồ thời gian thực DS3231. Quá trình hoạt động được chia thành hai giai đoạn: thiết lập thời gian cho ăn và kích hoạt quá trình cho ăn.

Trong giai đoạn thiết lập, khi người dùng thực hiện thao tác đặt thời gian, hàm **feeding_Handler()** trong vòng lặp chính phát hiện biến cờ **IsSetTime == 2**. Khi đó, hệ thống hiển thị thông báo "SETTING FEED TIME" trên màn hình LCD, đồng thời gọi hàm **DS3231_SetAlarm(_feed->alarm_time)** để lưu thời điểm báo thức tương ứng với giờ cho ăn vào bộ nhớ của mô-đun DS3231. Sau khi quá trình cài đặt hoàn tất, biến **IsSetTime** được đặt lại bằng 0 và LCD hiển thị "FEED TIME SAVED" để xác nhận người dùng đã đặt lịch thành công.

Ở giai đoạn tiếp theo, mô-đun DS3231 hoạt động độc lập để giám sát thời gian thực. Khi đến đúng thời điểm báo thức đã cài đặt, tín hiệu ngắt được gửi về hệ thống, kích hoạt quá trình cho ăn. LCD hiển thị thông báo "FEEDING START" và biến **IsFeed** được

gán giá trị 1, báo hiệu cho chương trình chính bắt đầu quy trình cấp thức ăn tự động thông qua điều khiển servo.



Hình 18: lưu đồ giải thuật mô tả hoạt động của mô hình cho cá ăn tự động

Module Đặt giờ cho ăn (Set Feeding Time)

- Khi nhấn nút 2 → giờ tăng lên.
- Khi nhấn nút 3 → giờ giảm xuống.
- Khi chọn được mốc giờ mong muốn → nhấn nút 1 để lưu lại.
- Mốc giờ này sẽ được đồng hồ DS3231 ghi nhớ (hoặc lưu vào biến RAM).
- Sau đó hệ thống quay lại vòng lặp chính.

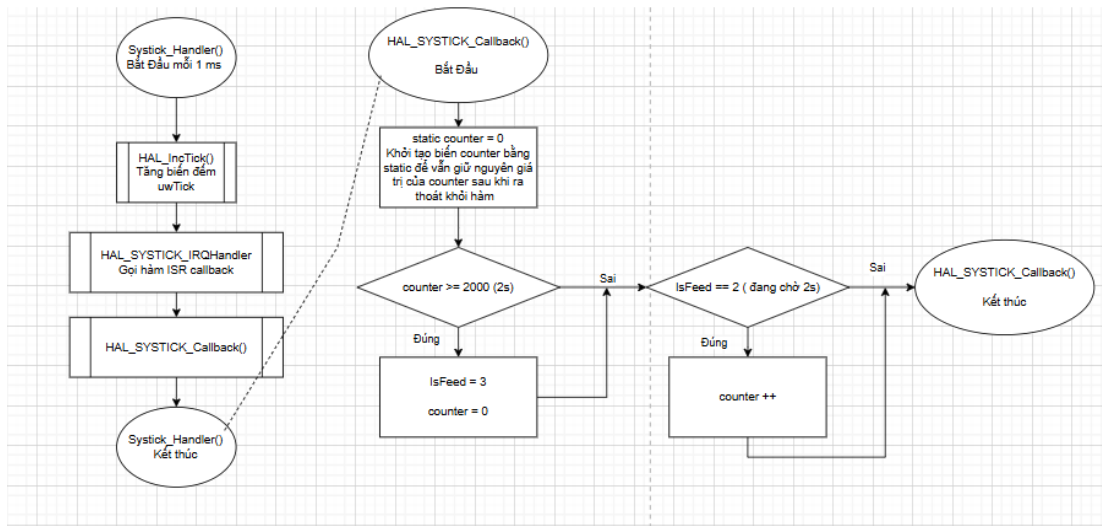
Module Cho ăn tự động (Auto Feeding)

- DS3231 cung cấp thời gian thực.
- Nếu giờ hiện tại == giờ đã set → Kích hoạt servo mở nắp thức ăn 2 giây.
- Servo đóng lại → Quay về vòng lặp chính.

Module Cho ăn thủ công (Manual Feeding)

- Khi nhấn nút 1 ở bất kỳ thời điểm nào → Servo mở 2 giây.
- Sau đó servo đóng lại → Quay lại vòng lặp chính.
- Module Đọc cảm biến & Hiện thị LCD

Bên cạnh lưu đồ chính thể hiện vòng lặp vô hạn và các chức năng tuần tự còn có 2 lưu đồ ngắt systick và ngắt ngoài và ngắt chỉ xảy ra bất chợt khi sự kiện đến, xử lý xong rồi quay về vòng lặp chính.



Hình 19: Lưu đồ mô tả cơ chế tạo thời gian chuẩn xác bằng ngắt SysTick để điều khiển servo cho ăn 2 giây.

d) Module siêu âm (đo lường thức ăn)

Khi hệ thống khởi động, chân Trigger của cảm biến siêu âm được kích ở mức cao trong một khoảng thời gian ngắn (khoảng 10 micro giây) để phát xung siêu âm ra môi trường. Sóng siêu âm này lan truyền trong không khí, gặp vật cản sẽ phản xạ lại và được chân Echo thu về.

Bộ vi điều khiển STM32 ghi nhận thời gian truyền và phản hồi của sóng siêu âm. Dựa vào thời gian này, vi điều khiển tính khoảng cách đến vật cản theo công thức:

$$\text{Distance} = \frac{\text{Time} \times 0.034}{2}$$

Sau khi tính toán xong, STM32 gửi dữ liệu khoảng cách đến ESP32 thông qua giao tiếp UART. Việc sử dụng UART giúp kết nối dễ dàng, tốc độ truyền cao, và tiêu tốn ít tài nguyên phần cứng. ESP32 nhận dữ liệu, chuyển đổi sang định dạng JSON và gửi lên máy chủ qua kết nối WiFi để lưu trữ và hiển thị.

2.4.4.2. ESP32 (Truyền dữ liệu)

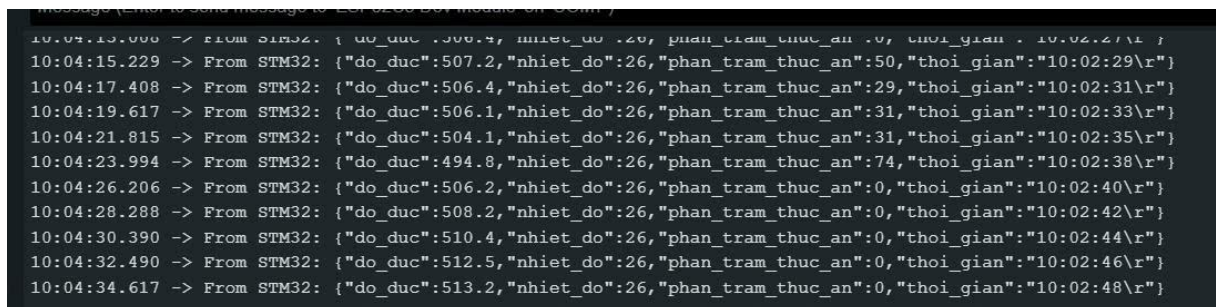
Vi điều khiển STM32 được sử dụng làm bộ xử lý trung tâm có nhiệm vụ thu thập dữ liệu từ các cảm biến. Sau khi xử lý và chuyển đổi tín hiệu cảm biến sang dạng số, STM32 sẽ truyền dữ liệu sang ESP32 thông qua giao tiếp UART (Serial) để tiếp tục xử lý và gửi lên nền tảng IoT. ESP32 trong hệ thống đóng vai trò là bộ truyền thông WiFi, chịu trách nhiệm đóng gói dữ liệu và gửi lên máy chủ MQTT hoặc các dịch vụ đám mây khác. Dữ liệu được STM32 truyền sang ESP32 ở dạng payload, tức là chuỗi dữ liệu thô bao gồm các giá trị đo được từ cảm biến. STM32 có thể gửi chuỗi "340,25.6\n" tương ứng với giá trị độ đục là 340 và nhiệt độ là 25.6°C. Dữ liệu này

không có cấu trúc cụ thể mà chỉ đơn giản là các giá trị được phân tách bằng dấu phẩy hoặc ký tự xuống dòng. Việc truyền payload giúp giảm kích thước gói tin, đảm bảo tốc độ nhanh và hạn chế độ trễ trong giao tiếp nối tiếp giữa hai vi điều khiển.

Sau khi ESP32 nhận được chuỗi payload, vi điều khiển này sẽ thực hiện phân tách và xử lý dữ liệu, sau đó đóng gói lại thành chuỗi JSON để gửi lên hệ thống MQTT. Việc sử dụng JSON ở giai đoạn này giúp dữ liệu có cấu trúc rõ ràng, dễ dàng xử lý ở phía server và thuận tiện cho việc hiển thị hoặc lưu trữ. Ví dụ, chuỗi payload "340,25.6\n" sau khi xử lý sẽ được ESP32 chuyển thành chuỗi JSON như sau:

```
{ "turbidity":340, "temperature":25.6 }
```

Chuỗi JSON này sau đó sẽ được xuất bản (publish) lên broker MQTT để gửi đến các thiết bị hoặc dịch vụ đăng ký nhận dữ liệu. Giao tiếp UART giữa STM32 và ESP32 được thiết lập theo chuẩn truyền 8 bit dữ liệu, còn gọi là cấu hình 8N1 (gồm 1 bit start, 8 bit dữ liệu, không parity và 1 bit stop). Điều đó có nghĩa là mỗi ký tự trong chuỗi payload đều được truyền đi dưới dạng 1 byte (8 bit). Quá trình truyền nhận diễn ra tuần tự, ESP32 đọc từng byte và ghép chuỗi lại để xử lý hoàn chỉnh. Tốc độ truyền (baud rate) thường được cài đặt ở các mức 9600 bps.



```
10:04:13.000 -> From STM32: { "do_duc":507.2, "nhiet_do":26, "phan_tram_thuc_an":50, "thoi_gian": "10:02:29\r"}
10:04:15.229 -> From STM32: { "do_duc":507.2, "nhiet_do":26, "phan_tram_thuc_an":50, "thoi_gian": "10:02:29\r"}
10:04:17.408 -> From STM32: { "do_duc":506.4, "nhiet_do":26, "phan_tram_thuc_an":29, "thoi_gian": "10:02:31\r"}
10:04:19.617 -> From STM32: { "do_duc":506.1, "nhiet_do":26, "phan_tram_thuc_an":31, "thoi_gian": "10:02:33\r"}
10:04:21.815 -> From STM32: { "do_duc":504.1, "nhiet_do":26, "phan_tram_thuc_an":31, "thoi_gian": "10:02:35\r"}
10:04:23.994 -> From STM32: { "do_duc":494.8, "nhiet_do":26, "phan_tram_thuc_an":74, "thoi_gian": "10:02:38\r"}
10:04:26.206 -> From STM32: { "do_duc":506.2, "nhiet_do":26, "phan_tram_thuc_an":0, "thoi_gian": "10:02:40\r"}
10:04:28.288 -> From STM32: { "do_duc":508.2, "nhiet_do":26, "phan_tram_thuc_an":0, "thoi_gian": "10:02:42\r"}
10:04:30.390 -> From STM32: { "do_duc":510.4, "nhiet_do":26, "phan_tram_thuc_an":0, "thoi_gian": "10:02:44\r"}
10:04:32.490 -> From STM32: { "do_duc":512.5, "nhiet_do":26, "phan_tram_thuc_an":0, "thoi_gian": "10:02:46\r"}
10:04:34.617 -> From STM32: { "do_duc":513.2, "nhiet_do":26, "phan_tram_thuc_an":0, "thoi_gian": "10:02:48\r" }
```

Hình 20: Hình ảnh gửi dữ liệu thành công từ STM32 đến ESP32 (8 bit)

Trong mô hình, ESP32 đóng vai trò là nút truyền thông trung gian, chịu trách nhiệm nhận dữ liệu từ vi điều khiển STM32 thông qua giao tiếp UART, sau đó truyền dữ liệu này lên máy chủ thông qua Wi-Fi và giao thức MQTT. ESP32 được kết nối với STM32 thông qua giao thức UART. Việc lựa chọn UART xuất phát từ ưu điểm dễ cấu hình, tốc độ truyền cao và tương thích với nhiều vi điều khiển, rất phù hợp cho việc truyền dữ liệu cảm biến hoặc lệnh điều khiển trong thời gian thực. Bên cạnh đó, giao thức MQTT được sử dụng vì đặc tính nhẹ, hiệu quả và phù hợp với các mạng có băng thông giới hạn. MQTT giúp giảm tải cho hệ thống trong khi vẫn duy trì khả năng truyền nhận dữ liệu ổn định giữa các thiết bị. So với các lựa chọn khác như ESP8266, ESP32

có ưu thế về hiệu năng, hỗ trợ nhiều giao thức truyền thông, và dễ dàng lập trình. Nhờ đó, việc tích hợp ESP32 giúp hệ thống đạt được tính ổn định, linh hoạt và mở rộng tốt hơn trong tương lai.

```
11:58:16.529 ->
11:58:16.529 -> === AquaNova ESP32 MQTT ===
11:58:16.529 -> [WiFi] Connecting to Wifi
11:58:16.529 -> ...
11:58:16.529 -> [WiFi] Connected!
11:58:16.529 -> [WiFi] IP: 10.183.170.196
11:58:16.529 -> [NTP] Syncing time...
11:58:16.853 -> [NTP] Time synced successfully!
11:58:16.853 -> [NTP] Current Time: 11:58:35
11:58:16.853 -> [SYSTEM] Setup complete!
11:58:16.897 -> [NTP] Re-syncing time before MQTT connection...
11:58:16.897 -> [MQTT] Attempting to connect... (1/5)
11:58:20.047 -> [MQTT] Connected!
11:58:20.047 -> [MQTT] Subscribed to: aquanova/devices/control
```

Hình 21: Kết nối Wifi và đồng bộ thời gian

Hình ảnh minh chứng cho thấy ESP32 đã khởi tạo và kết nối thành công với mạng Wi-Fi 2.4 GHz, với địa chỉ IP nội bộ được cấp là 10.183.170.196. Sau khi kết nối, module tiến hành đồng bộ thời gian thực (NTP sync) đảm bảo tính chính xác của các gói tin, đặc biệt trong ứng dụng ghi nhận dữ liệu theo thời gian thực.

2.4.5. Thiết kế web

Hệ thống AquaNova được tổ chức theo kiến trúc 3 lớp (Three-Tier Architecture) kết hợp mô hình MVC (Model – View – Controller), bao gồm:

Lớp	Chức năng	Thành phần trong dự án
Presentation Layer (Giao diện người dùng)	Hiển thị dữ liệu cảm biến, cho phép người dùng điều khiển, đặt lịch cho hệ thống cho ăn. Toàn bộ giao diện được xây dựng bằng HTML, CSS và JavaScript.	Thư mục templates/ (các file .html), static/css/, static/js/
Application Layer (Xử lý trung gian)	Xử lý logic ứng dụng, kết nối cơ sở dữ liệu và MQTT. Sử dụng Flask Framework để định tuyến request và phản hồi dữ liệu JSON cho frontend.	File app.py và các module trong blueprints/ gồm dashboard, control, telemetry, admin

Data Layer (Lưu trữ và giao tiếp IoT)	Lưu trữ dữ liệu thời gian thực, bao gồm giá trị cảm biến, lịch cho ăn, log hoạt động. Dữ liệu được lưu trên Firebase Firestore và giao tiếp cảm biến qua MQTT Broker.	firebase_admin_init.py, collection readings, schedules, feed_logs
--	---	---

Cấu trúc thư mục chính:

aquanova/	
├── app.py	# Khởi tạo Flask app, load config, đăng ký blueprint
├── blueprints/	
│ ├── dashboard/routes.py	# Cung cấp API dữ liệu (temperature, turbidity, feed)
│ ├── control/routes.py	# Điều khiển cho ăn, tạo/xóa lịch
│ ├── control/scheduler.py	# Lập lịch tự động kiểm tra và gửi lệnh feed
│ ├── telemetry/routes.py	# Ghi nhận dữ liệu từ MQTT (telemetry)
│ ├── admin/routes.py	# Giao diện quản trị
├── firebase_admin_init.py	# Kết nối Firestore
├── mqtt/listener.py	# MQTT subscriber nhận dữ liệu cảm biến
├── mqtt/	
│ ├── listener.py	# sub
│ └── publisher.py	
├── static /	
│ ├── css/styles.css	# Giao diện hiển thị
│ └── js/dashboard.js	# Xử lý cập nhật dữ liệu & biểu đồ
├── templates/	
│ ├── index.html	# Dashboard tổng quan
│ ├── feedtimer.html	# Giao diện đặt lịch cho ăn
│ ├── temperature.html	# Biểu đồ nhiệt độ
│ ├── turbidity.html	# Biểu đồ độ đục
│ └── admin.html	# Trang quản trị

Chuẩn bị môi trường

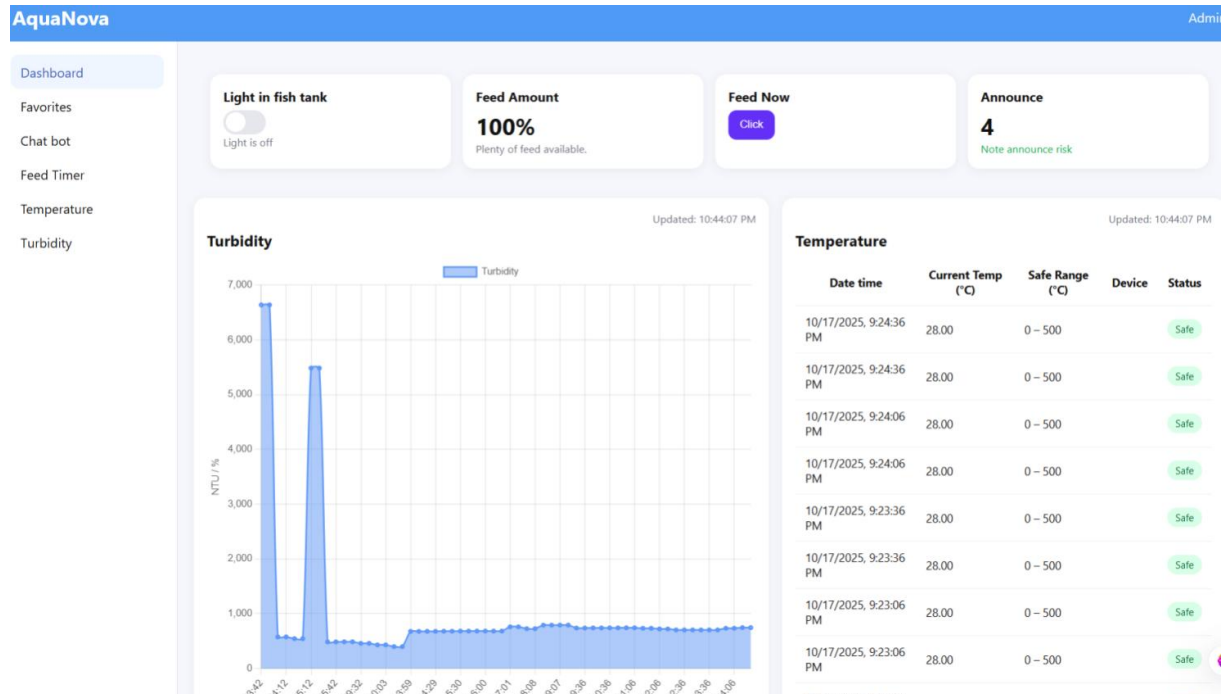
Database
FIREBASE_CREDENTIALS=C:\Users\ASUS\Documents\GitHub\IOT_AquaNova\IoT_AquaNova\serviceAccount.json
FIREBASE_DB=firestore
SECRET_KEY=AquaNovaSecret123
HiveMQ Cloud MQTT
MQTT_HOST=d2b2655505394c9b8e88c06c8d30d912.s1.eu.hivemq.cloud
MQTT_PORT=8883
MQTT_USER=aquanova_user
MQTT_PASS=12345678Aqua
MQTT_TOPIC=aquanova/telemetry

2.5.1. Frontend

Frontend giao tiếp trực tiếp với Flask qua các API REST, sử dụng Fetch API để lấy dữ liệu dạng JSON. Các biểu đồ hiển thị dữ liệu thời gian thực được dựng bằng Chart.js

a) Cấu trúc chung của giao diện

Tất cả các trang đều tuân theo bố cục 2 phần chính:



Hình 22: Bố cục giao diện của trang web người dùng

- Thanh bên trái (sidebar / navigation): chứa các liên kết tới các module chính (Dashboard, Temperature, Turbidity, Feed Timer, Admin).
- Khu vực nội dung chính (main): hiển thị dữ liệu, biểu đồ và bảng tương ứng.

Ví dụ đoạn HTML cấu trúc tổng quát:

```
<div class="layout">
  <aside>
    <nav class="menu">
      <a href="/dashboard" class="tab-link active">Dashboard</a>
      <a href="/temperature" class="tab-link">Temperature</a>
      <a href="/turbidity" class="tab-link">Turbidity</a>
      <a href="/feedtimer" class="tab-link">Feed Timer</a>
    </nav>
  </aside>
  <main>
    <!-- Nội dung từng module -->
  </main>
</div>
```

Mỗi trang đều dùng tệp styles.css để thống nhất màu chủ đạo:

```
:root {
  --brand: #0ea5e9; /* Xanh AquaNova */
  --muted: #6b7280; /* Ghi nhạt */
  --bg: #f6f7fb; /* Nền sáng */
}
.card {
  background: #fff;
  border-radius: 14px;
  box-shadow: 0 2px 12px rgba(0,0,0,.05);
}
```

b) Dashboard – Trang tổng quan (index.html)

- Trang Dashboard là trung tâm giám sát mô hình. Chức năng chính:
- Hiển thị các thông số tổng quan:
- Lượng thức ăn còn lại (%)
- Độ đục nước (NTU)
- Nhiệt độ trung bình (°C)
- Số lần cảnh báo (Announce)
- Biểu đồ đường (line chart) thể hiện độ đục nước theo thời gian.
- Bảng dữ liệu nhiệt độ: 10 giá trị gần nhất kèm đánh giá “Safe” hoặc “Alert”.

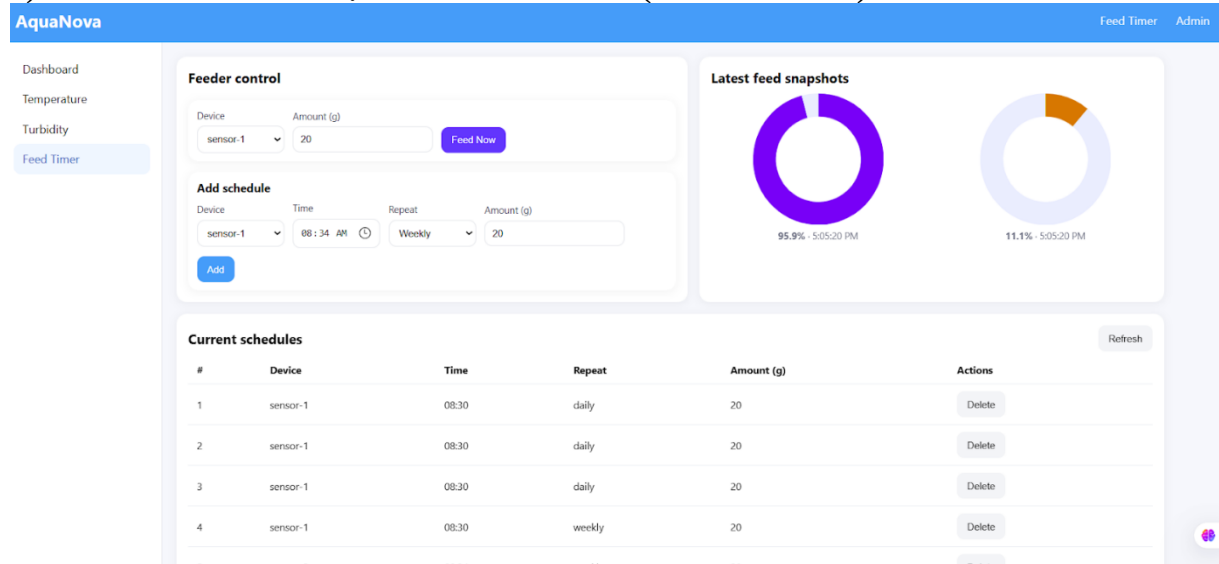
Cơ chế cập nhật: File static/js/dashboard.js thực hiện gọi API định kỳ (5s/lần):

```
async function loadAll(){
  const latest = await (await fetch('/dashboard/latest?n=60')).json();
  const items = latest.items || [];
  const arr = items.slice().reverse();
  const labels = arr.map(r => new Date(r.ts).toLocaleTimeString());
  const turb = arr.map(r => r.turbidity);
  ensureChart(document.getElementById('turbChart').getContext('2d'), labels,
turb);
}
setInterval(loadAll, 5000);
```

API Flask /dashboard/latest trả về JSON:

```
{
  "items": [
    { "ts": "2025-10-17T14:21:36Z", "temperature": 28, "turbidity": 733.7,
"feed": 100}
  ]
}
```

c) Feed Timer – Giao diện điều khiển cho ăn (feedtimer.html)



1. **Feed Now** — gửi lệnh cho ăn ngay lập tức qua MQTT (qua Flask API /control/feed-now):

```
await fetch('/control/feed-now', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ amount: 20 })
});
```

→ Flask publish đến topic aquanova/control.

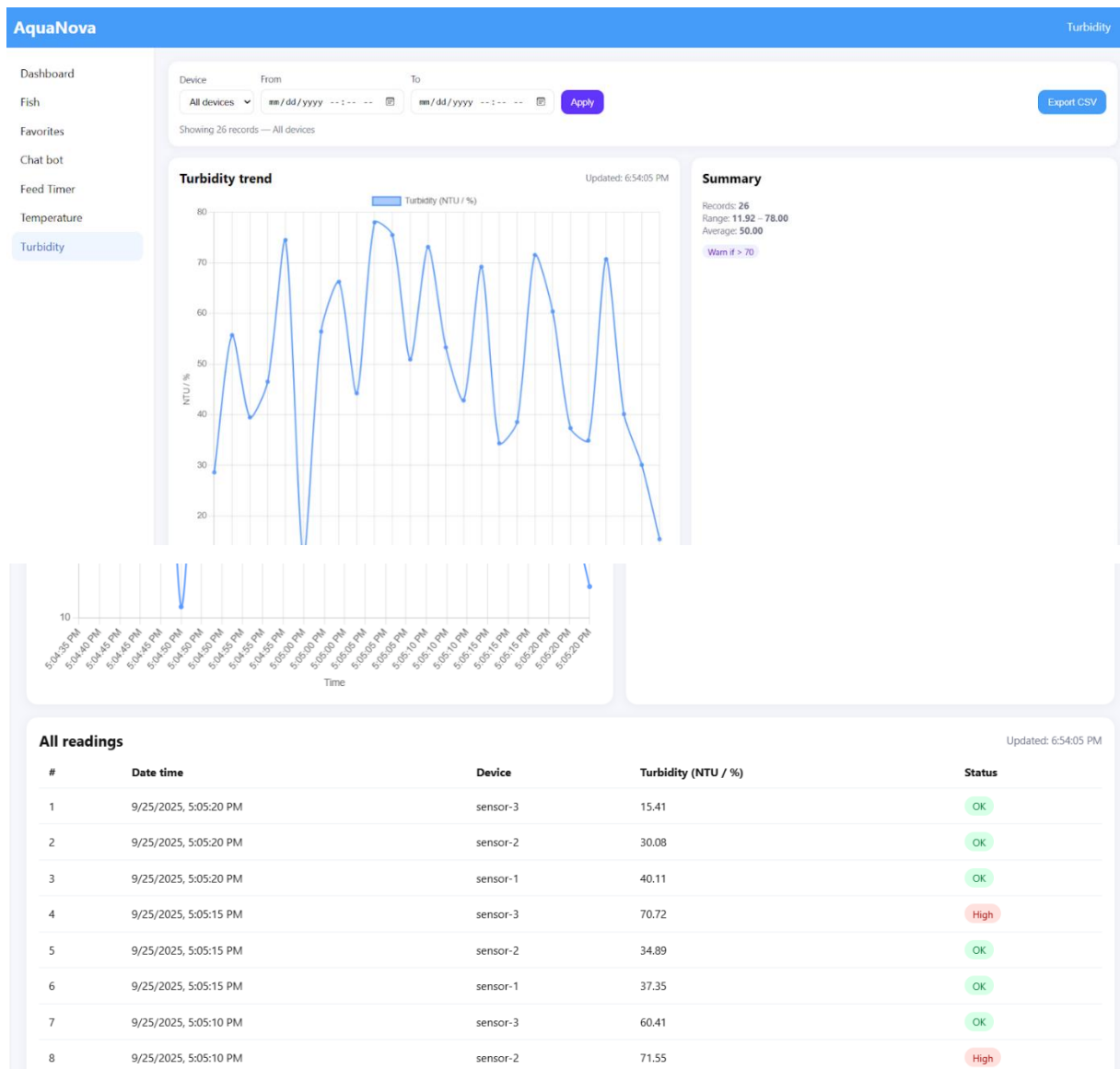
- Add Schedule — đặt lịch cho ăn theo ngày, giờ, và kiểu lặp (Once, Daily, Weekly).
- Current Schedules — hiển thị bảng các lịch đã đặt, lấy từ API /control/schedules.

Hiển thị biểu đồ phần trăm thức ăn: biểu đồ tròn (doughnut) hiển thị % thức ăn còn lại, sử dụng Chart.js:

```
pieCharts[canvasId] = new Chart(ctx, {
  type: 'doughnut',
  data: {
    labels: ['Remaining', 'Used'],
    datasets: [{ data: [80, 20], backgroundColor: ['#4f46e5', '#e8eefb'] }]
  }
});
```

d) Temperature – Giám sát nhiệt độ

- Biểu đồ line chart hiển thị biến động nhiệt độ.
- Bảng dưới biểu đồ hiển thị timestamp, giá trị, thiết bị, trạng thái



e) Turbidity – Giám sát độ đục nước

- Biểu đồ đường tương tự Temperature, nhưng lấy dữ liệu turbidity.
- Khi giá trị vượt 1000 NTU, phần tử đổi sang class alert để cảnh báo.

h) Tương tác giữa Front end với Flask API

Hoạt động	API Flask	Phương thức	Kết quả
Lấy dữ liệu cảm biến	/dashboard/latest	GET	Trả về danh sách các bản ghi mới nhất
Cho ăn ngay lập tức	/control/feed-now	POST	Gửi lệnh MQTT → thiết bị thực thi
Tạo lịch cho ăn	/control/schedule	POST	Lưu thông tin lịch vào Firestore

Lấy danh sách lịch	/control/schedules	GET	Hiển thị bảng “Current Schedules”
Xóa lịch	/control/schedules/<id>	DELETE	Xóa lịch tương ứng khỏi Firestore

2.5.2. Backend

Mô hình được xây dựng trên nền Flask Framework (Python) với mô hình MVC mở rộng bằng Blueprint. Kiến trúc tổng thể gồm các thành phần chính:

- Flask Server: đóng vai trò trung tâm, nhận yêu cầu từ giao diện web, xử lý và tương tác với cơ sở dữ liệu.
- Firestore Database: lưu trữ dữ liệu cảm biến (nhiệt độ, độ ẩm, phần trăm thức ăn) và các lịch cho ăn tự động.
- MQTT Broker (HiveMQ Cloud): làm cầu nối giữa thiết bị IoT (ESP32) và server. Flask vừa publish lệnh điều khiển, vừa subscribe dữ liệu telemetry.
- Scheduler (APScheduler): bộ lập lịch nền, kiểm tra các lịch hẹn từ Firestore và gửi lệnh điều khiển đúng thời điểm.
- Frontend Web: giao diện hiển thị dữ liệu và điều khiển, giao tiếp với Flask qua các REST API.

a) Cấu trúc Flask và Blueprints

Blueprint	Mục đích	API chính
dashboard_bp	Cung cấp dữ liệu hiển thị cho Dashboard	/dashboard/latest, /dashboard/last, /dashboard/announce-count
control_bp	Cho phép điều khiển cho ăn và quản lý lịch	/control/feed-now, /control/schedule, /control/schedules
telemetry_bp	Nhận dữ liệu từ thiết bị IoT qua MQTT	/telemetry/upload (hoặc auto listener)
admin_bp	Trang quản trị hệ thống	/admin

b) Kết nối cơ sở dữ liệu Firestore

Firestore được chọn vì khả năng đồng bộ thời gian thực, dễ mở rộng và hỗ trợ tốt với Python thông qua firebase_admin. Khởi tạo trong file firebase_admin_init.py:

```
import firebase_admin
from firebase_admin import credentials, firestore
from config import Config
```

```
cred = credentials.Certificate(Config.FIREBASE_CREDENTIALS)
firebase_admin.initialize_app(cred)
db = firestore.client()
```

Dữ liệu được tổ chức theo các collection:

- readings/ – lưu dữ liệu cảm biến (temperature, turbidity, feed, ts).
- schedules/ – lưu các lịch cho ăn tự động (date, time, repeat, amount).
- feed_logs/ – lưu lịch sử cho ăn thực tế.

c) Module Dashboard (Giao tiếp với giao diện)

dashboard/routes.py định nghĩa các API RESTful để web có thể lấy dữ liệu cảm biến và thống kê tổng quan.

```
@dashboard_bp.get("/latest")
def latest():
    docs = (db.collection("readings")
            .order_by("ts", direction=firestore.Query.DESENDING)
            .limit(100).stream())
    items = [{"ts": d.get("ts"),
              "temperature": d.get("temperature"),
              "turbidity": d.get("turbidity"),
              "feed": d.get("feed")} for d in docs]
    return jsonify({"items": items})
```

d) Module Control (điều khiển từ Web)

File control/routes.py chịu trách nhiệm nhận yêu cầu điều khiển từ web và gửi lệnh qua MQTT.

Cấu trúc gồm hai nhóm API:

- Cho ăn ngay lập tức

```
@control_bp.post("/feed-now")
def feed_now():
    data = request.get_json(force=True)
    amount = data.get("amount", 20)
    topic = "aquanova/control"
    payload = {"cmd": "feed", "amount": amount}
    _get_pub().publish(topic, json.dumps(payload), qos=1)
    db.collection("feed_logs").add({
        "amount": amount,
        "timestamp": firestore.SERVER_TIMESTAMP
    })
    return jsonify({"ok": True})
```

Khi người dùng bấm “Feed Now” trên web, lệnh được gửi tới MQTT Broker qua topic aquanova/control với mức QoS = 1, đảm bảo lệnh không bị mất.

- Đặt lịch tự động

```

@control_bp.post("/schedule")
def add_schedule():
    data = request.get_json(force=True)
    item = {
        "id": uuid.uuid4().hex[:12],
        "date": data.get("date"),
        "time": data.get("time"),
        "repeat": data.get("repeat"),
        "amount": data.get("amount")
    }
    db.collection("schedules").document(item["id"]).set(item)
    return jsonify({"ok": True, "item": item})

```

Mã nhận thành công

```

127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now HTTP/1.1" 200 -
{'cmd': 'feed', 'amount': 10}

```

e) Bộ lập lịch tự động (Scheduler)

Module scheduler.py sử dụng BackgroundScheduler của APScheduler để kiểm tra lịch cho ăn mỗi phút. Nguyên lý hoạt động:

- Lấy toàn bộ document trong collection schedules (Firestore).
- Kiểm tra date, time, repeat (none, daily, weekly).
- Nếu khớp thời gian hiện tại → gửi lệnh MQTT:

```

payload = {"cmd": "feed", "amount": amount}
_get_pub().publish("aquanova/control", json.dumps(payload), qos=1)

```

Sử dụng QoS = 1 vì :

- Flask sẽ chờ xác nhận (PUBACK) từ Broker trước khi coi lệnh “đã gửi thành công”.
- Nếu mất gói hoặc timeout → Flask tự động gửi lại. Trên Broker, MCU sẽ nhận được ít nhất 1 bản sao của lệnh. Điều này rất quan trọng để đảm bảo lệnh cho ăn không bị mất, ngay cả khi mạng bị trễ hoặc mất tạm thời.

Lưu log vào feed_logs/.

Nếu repeat == "none" → tự xóa sau khi chạy.

[SCHEDULE] Feed 20g → aquanova/control (repeat=daily)

2.5.3. MQTT (publish, subscriber)

Mô hình AquaNova có cấu trúc gồm nhiều thành phần:

- Thiết bị vi điều khiển (ESP32/STM32) thu thập dữ liệu cảm biến (nhiệt độ, độ ẩm) và gửi về server.
- Server Flask nhận dữ liệu qua Broker và lưu vào Firestore.
- Web dashboard hiển thị dữ liệu thời gian thực.
- Scheduler tự động gửi lệnh “Feed” định kỳ qua MQTT.

Với mô hình này cần:

- Giao thức có cơ chế đảm bảo tin cậy cho lệnh điều khiển.
- Hỗ trợ giao tiếp hai chiều (thiết bị ↔ server).
- Dễ tích hợp với Python (Flask) và ESP32 (PubSubClient).
- Có thể mở rộng cho nhiều hồ cá hoặc nhiều thiết bị song song.

Kết luận: MQTT phù hợp hơn nhờ mô hình Publish/Subscribe linh hoạt. Server chỉ cần publish lệnh đến topic aquanova/control, mọi thiết bị đăng ký (subscribe) đều nhận được. CoAP chỉ phù hợp nếu có một số lượng nhỏ thiết bị và yêu cầu giao tiếp trực tiếp dạng truy vấn REST. Trong trường hợp nhiều thiết bị cùng hoạt động, CoAP khó mở rộng và không có kênh trung gian quản lý.

- Publisher: Flask server (hoặc Scheduler) gửi lệnh điều khiển cho thiết bị.
- Subscriber: Flask server đồng thời lắng nghe (subscribe) dữ liệu cảm biến từ thiết bị.
- Broker: HiveMQ Cloud – chịu trách nhiệm trung chuyển dữ liệu giữa các node

a. MQTT Listener (Subscriber)

Tệp mqtt/listener.py chịu trách nhiệm kết nối, đăng ký (subscribe) và xử lý dữ liệu cảm biến.

Sau khi Flask khởi động, hàm start_mqtt_background() được gọi trong luồng nền:

```
def start_mqtt_background(app):
    client = mqtt.Client(
        mqtt.CallbackAPIVersion.VERSION2,
        client_id=f"aquanova-server-{uuid.uuid4().hex[:6]}"
    )
    client.username_pw_set(cfg["MQTT_USER"], cfg["MQTT_PASS"])
    client.tls_set()
    client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]), keepalive=60)
    client.on_message = on_message
    client.subscribe(cfg["MQTT_TOPIC"], qos=1)
    client.loop_forever()
```

Khi nhận bản tin từ thiết bị:

```
def on_message(client, userdata, msg):
    payload = msg.payload.decode("utf-8")
    data = json.loads(payload)
    data.setdefault("ts", datetime.now(timezone.utc).isoformat())
    db.collection("readings").add(data)
```

b. MQTT Publisher (Flask gửi lệnh điều khiển)

File mqtt/publisher.py định nghĩa lớp MQTTPublisher, chịu trách nhiệm publish lệnh đến thiết bị IoT.

Khởi tạo và kết nối:

```
self.client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id=f"aquanova-publisher-{datetime.now().timestamp()}"
)
self.client.username_pw_set(user, pwd)
self.client.tls_set()
self.client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]), keepalive=60)
self.client.loop_start()
```

Gửi lệnh điều khiển:

```
def publish(self, topic: str, payload: dict, qos: int = 1):
    msg = json.dumps(payload)
    result = self.client.publish(topic, msg, qos=qos)
    result.wait_for_publish()
```

Việc gọi wait_for_publish() đảm bảo gói tin được gửi hoàn tất trước khi tiếp tục xử lý, giúp hệ thống ổn định hơn.

Trong AquaNova, **QoS = 1** được chọn cho cả subscribe và publish:

```
client.subscribe(cfg["MQTT_TOPIC"], qos=1)
client.publish("aquanova/control", msg, qos=1)
```

Điều này đảm bảo rằng:

- Dữ liệu hoặc lệnh sẽ đến ít nhất một lần, không bị mất.
- Flask và ESP32 đều nhận được xác nhận (PUBACK) từ Broker.
- Nếu có lỗi mạng, client sẽ tự gửi lại cho đến khi thành công.

2.4.6. Kết hợp Machine Learning

Mục tiêu: Dự đoán các hành vi sinh học của cá (như thời điểm thay nước, lượng thức ăn cần thiết) Từ đó, tối ưu hóa hoạt động cho ăn tự động.

- Cơ chế thu thập dữ liệu
 - Tiền xử lý dữ liệu
 - Mô hình học máy sử dụng
- Random Forest Regressor (mô hình hồi quy)

d) Công thức và ví dụ dự đoán

- Dự đoán thời điểm thay nước
- Dự đoán lượng thức ăn hợp lý từ lịch sử các lần cho ăn
- Dự đoán tuổi thọ trung bình của cá

2.4.7. Hiệu chuẩn và kiểm thử (to be continue)

a) Đánh giá độ chính xác dữ liệu (môi trường + thiết bị đo)

b) Đánh giá đường truyền wifi (kết nối)

c) Reconnect MQTT

Giao thức MQTT hoạt động dựa trên kết nối TCP/IP lâu dài (persistent connection) giữa Client (ESP32 hoặc Flask) và Broker (HiveMQ Cloud).

Kết nối này phải được duy trì liên tục để:

- nhận các bản tin mới (đối với Subscriber),
- hoặc gửi lệnh điều khiển (đối với Publisher).

Nếu vì lý do nào đó kết nối bị ngắt mà không được tự khôi phục, client sẽ mất toàn bộ dữ liệu hoặc lệnh điều khiển trong khoảng thời gian đó.

- Phía web server: trong listener.py, hàm callback on_disconnect() được định nghĩa để xử lý ngay khi mất kết nối:

```
def on_disconnect(client, userdata, flags, reason_code, properties=None):
    write_log(f"[MQTT] DISCONNECTED (code={reason_code}) - attempting
reconnect...")
    while True:
        try:
            client.reconnect()
            write_log("[MQTT] RECONNECTED successfully!")
            break
        except Exception as e:
            time.sleep(5)
```

- Phía MCU:...

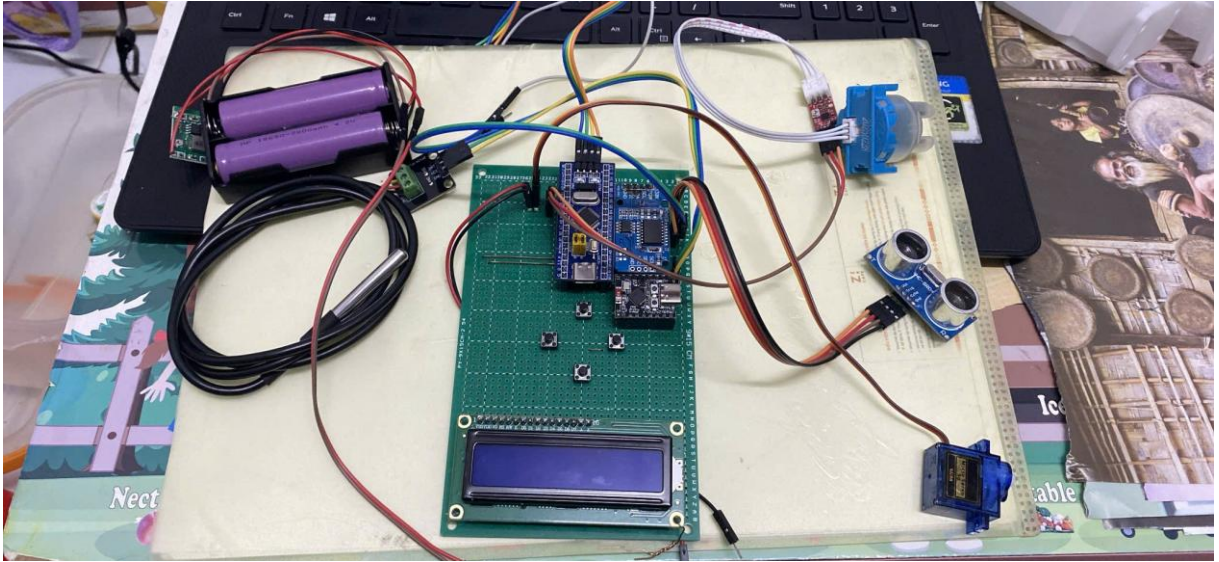
d) Độ trễ và tần suất cập nhật

CHƯƠNG 3

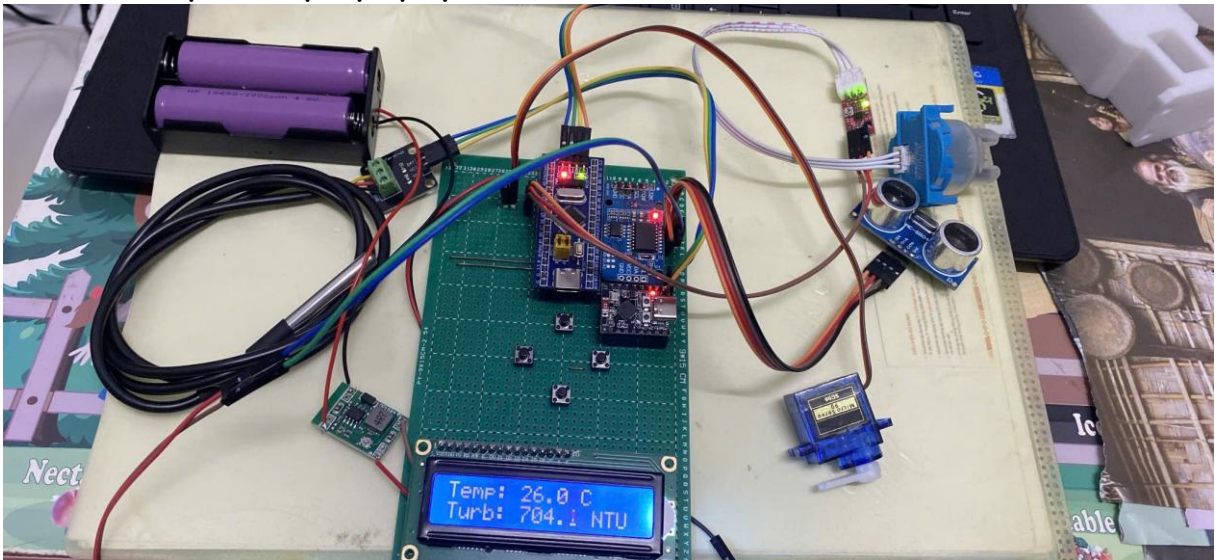
THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ

3.1. Kết quả thiết kế phần cứng

Hình: phần cứng toàn bộ



Hình: hiển thị lcd nhiệt độ độ đục



Hình : hiển thị set tgian cho ăn + Hình: feeding

Hình: hiển thị lượng thức ăn

Hình nhận tín hiệu đèn bật tắt

Hình cảnh báo vượt ngưỡng

3.1. Kết quả thiết kế phần mềm

- Log MQTT (CONNECT, PUBLISH, SUBACK).

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5000

Press CTRL+C to quit

* Restarting with stat

[Firebase Init] Using credentials: C:\Users\ASUS\Documents\GitHub\IoT_AquaNova\IoT_AquaNova\serviceAccount.json

[Firebase Init] Using credentials: C:\Users\ASUS\Documents\GitHub\IoT_AquaNova\IoT_AquaNova\serviceAccount.json

2025-10-18T02:39:51.821393 [MQTT] Connecting to fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud:8883

2025-10-18T02:39:51.821433 [MQTT] LISTENING fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud:8883 topic=aquanova/devices/telemet

[Scheduler] Started - checking schedules every minute

* Debugger is active!

* Debugger PIN: 177-325-123

2025-10-18T02:39:52.077474 [MQTT] CONNECTED with code=Success

2025-10-18T02:39:52.078805 [MQTT] SUBSCRIBE sent topic=aquanova/devices/telemetry mid=1 result=0

2025-10-18T02:39:52.335059 [MQTT] SUBACK mid=1, granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]

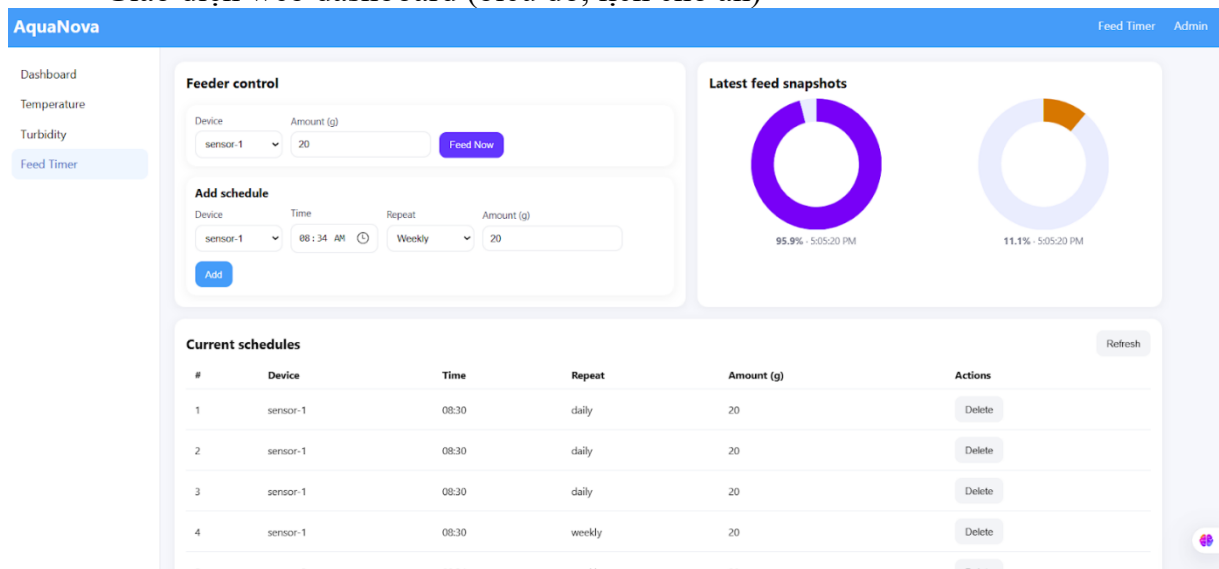
2025-10-18T02:39:53.355134 [MQTT] CONNECTED with code=Success

2025-10-18T02:39:53.356642 [MQTT] SUBSCRIBE sent topic=aquanova/devices/telemetry mid=1 result=0

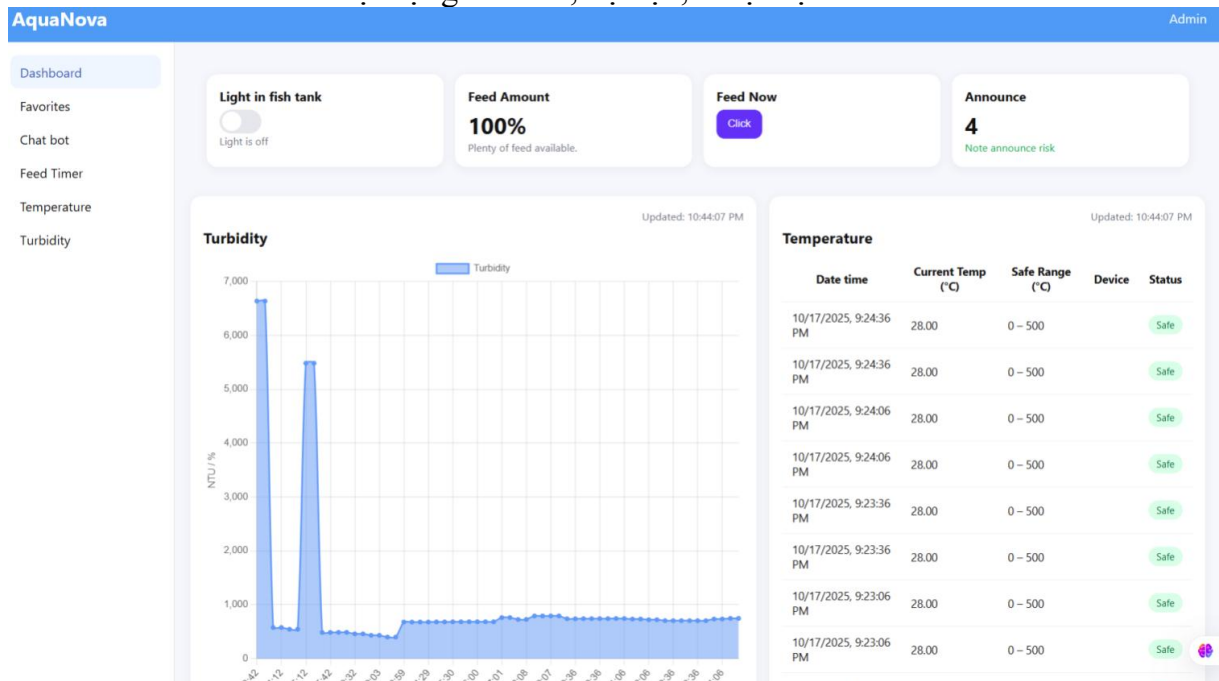
2025-10-18T02:39:55.665989 [MQTT] SUBACK mid=1, granted_qos=[ReasonCode(Suback, 'Granted QoS 1')]

]]

- Giao diện web dashboard (biểu đồ, lịch cho ăn)



- Dashboard hiển thị lượng thức ăn, độ đục, nhiệt độ



- Gửi tin thực tế hiển thị trên HiveMQ và Firebase.

Subscriber

```
, 'ts': '2025-10-16T04:54:42.981082+00:00'}
127.0.0.1 - - [16/Oct/2025 11:54:47] "GET /feedtimer HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 11:54:49] "GET /static/css/styles.css HTTP/1.1" 304 -
127.0.0.1 - - [16/Oct/2025 11:54:49] "GET /static/img/logo_aquanova.png HTTP/1.1" 304 -
127.0.0.1 - - [16/Oct/2025 11:54:49] "GET /control/schedules HTTP/1.1" 200 -
2025-10-16T11:55:13.108757 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 5483.8, 'temperature': 26, 'feed': 100, 'thoi_gian': '11:52:10', 'ts': '2025-10-16T04:55:12.838187+00:00'}
2025-10-16T11:55:13.124161 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 5483.8, 'temperature': 26, 'feed': 100, 'thoi_gian': '11:52:10', 'ts': '2025-10-16T04:55:12.835729+00:00'}
2025-10-16T11:55:43.206283 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 485.1, 'temperature': 26, 'feed': 100, 'thoi_gian': '11:52:39', 'ts': '2025-10-16T04:55:42.928803+00:00'}
2025-10-16T11:55:43.223584 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 485.1, 'temperature': 26, 'feed': 100, 'thoi_gian': '11:52:39', 'ts': '2025-10-16T04:55:42.919996+00:00'}
2025-10-16T11:56:13.117407 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 486.8, 'temperature': 26, 'feed': 10494, 'thoi_gian': '26.0,100,11:53:10', 'ts': '2025-10-16T04:56:12.846771+00:00'}
```

Listener

```
{'cmd': 'feed', 'amount': 10}
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now HTTP/1.1" 200 -
{'cmd': 'feed', 'amount': 10}
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:12] "POST /control/feed-now HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:12] "POST /control/feed-now HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:12] "POST /control/feed-now HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:12] "POST /control/feed-now HTTP/1.1" 200 -
2025-10-16T12:00:33.326305 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 25-10-16T05:00:33.003208+00:00'}
2025-10-16T12:00:33.346791 [MQTT] MESSAGE topic=aquanova/devices/telemetry payload={'turbidity': 25-10-16T05:00:33.013879+00:00'}
127.0.0.1 - - [16/Oct/2025 12:00:35] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:36] "GET /static/css/styles.css HTTP/1.1" 304 -
127.0.0.1 - - [16/Oct/2025 12:00:36] "GET /dashboard/last HTTP/1.1" 200 -
127.0.0.1 - - [16/Oct/2025 12:00:36] "GET /static/img/logo_aquanova.png HTTP/1.1" 304 -
```

3.2. Mô phỏng một số kịch bản

Để kiểm chứng hoạt động của hệ thống, nhóm tiến hành mô phỏng một số kịch bản thực tế nhằm đánh giá khả năng đo đạc, phản hồi và điều khiển tự động của mô hình Kịch bản 1 – Kiểm thử cảm biến độ đục (đang thực hiện)

Mục tiêu: Đánh giá khả năng phát hiện và hiển thị sự thay đổi độ đục của nước.

Thực hiện:

- Bơm nước sạch vào hồ (nước trong).
- Ghi nhận giá trị cảm biến hiển thị trên LCD và web dashboard.
- Thêm các hạt cặn hoặc bột vào hồ để tạo độ đục nhân tạo.
- Quan sát sự thay đổi giá trị đo được từ cảm biến và phản hồi trên giao diện.

Kết quả mong đợi:

Trạng thái nước	Giá trị đo (NTU)	Hiển thị LCD	Kết luận
Nước trong	0–100 NTU	“Turb: 45.3 NTU”	
Nước đục	>700 NTU	“Turb: 764.1 NTU”	

Đánh giá:

Kịch bản 2 – Kiểm thử cảm biến độ đục (đang thực hiện)

Mục tiêu: Đánh giá độ chính xác và tính ổn định của cơ cấu định lượng thức ăn theo lịch trình hoặc thao tác thủ công.

Thực hiện:

- Thiết lập lượng thức ăn 5g/lần thông qua web dashboard.
- Đặt lịch cho ăn tự động vào thời điểm cố định trong ngày (ví dụ: 8h sáng).
- Quan sát servo quay và lượng thức ăn rơi vào bể cá.
- Lặp lại trong 3 ngày liên tiếp để đánh giá độ ổn định

Kết quả mong đợi:

Ngày	Lệnh thực hiện	Lượng thức ăn thực tế (g)	Trạng thái	Ghi chú
1	Tự động			
2	Tự động			
3	Điều khiển thủ công			

Đánh giá:

3.3. Nhận xét, đánh giá và kết quả

Trong suốt quá trình nghiên cứu, thiết kế, lập trình và thi công mô hình hệ thống đo độ đục và quản lý bể cá thông minh, tôi đã gặp không ít khó khăn, từ việc lựa chọn cảm biến phù hợp, thiết kế mạch phần cứng, đến xử lý truyền thông giữa STM32 và ESP32. Nhìn chung, hệ thống đã thể hiện được tính khả thi, hoạt động đúng theo nguyên lý và có tiềm năng phát triển thành một sản phẩm ứng dụng thực tế. Tuy nhiên, bên cạnh những ưu điểm đạt được, mô hình vẫn tồn tại một số hạn chế nhất định cần được cải thiện trong tương lai.

Ưu điểm:

- e) Hệ thống vận hành ổn định, đọc và xử lý dữ liệu độ đục chính xác, phản hồi nhanh.
- f) Kết nối UART giữa STM32 và ESP32 đơn giản, truyền dữ liệu ổn định, ít lỗi.
- g) Giao tiếp dữ liệu qua MQTT giúp dễ dàng giám sát từ xa, tiện cho ứng dụng IoT.
- h) Cấu trúc mạch gọn gàng, chi phí thi công thấp hơn nhiều so với các hệ thống thương mại có cùng chức năng.

- i) Có khả năng mở rộng thêm nhiều cảm biến khác (pH, DO, mực nước) và điều khiển thiết bị như bơm, quạt, hoặc máy lọc nước.
- j) Tự động hóa được các tác vụ như cho ăn, lọc và thay nước theo độ đục hoặc thời gian định sẵn, giảm thao tác thủ công.

Nhược điểm:

- k) Hệ thống chưa có cơ chế mã hóa hoặc xác thực dữ liệu, dẫn đến tính bảo mật chưa cao.
- l) Cảm biến độ đục dễ bị ảnh hưởng bởi môi trường (ánh sáng ngoài, cặn, rêu bám), cần hiệu chuẩn định kỳ.
- m) Chưa có khả năng tự động kết nối lại WiFi nếu mất mạng, cần reset thủ công để khởi động lại ESP32.
- n) Giao diện giám sát còn đơn giản, chưa có cảnh báo thông minh hoặc lưu trữ dữ liệu dài hạn.
- o) Thuật toán xử lý còn cơ bản, chưa tối ưu về thời gian và khả năng phân tích dữ liệu môi trường.

Đánh giá: qua quá trình thử nghiệm thực tế, mô hình đã đáp ứng được mục tiêu đề ra: đo và hiển thị giá trị độ đục, truyền dữ liệu ổn định và điều khiển các thiết bị trong hệ thống. Các chức năng cơ bản đều hoạt động đúng yêu cầu, cho thấy thiết kế mạch và phần mềm có tính ổn định cao. Tuy vậy, để mô hình trở thành một sản phẩm hoàn chỉnh và ứng dụng được trong thực tế, cần tiếp tục nâng cấp về bảo mật, khả năng tự động hóa, giao diện giám sát và độ chính xác của cảm biến.

CHƯƠNG 4

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

To be continue

4.2. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng và hoàn thiện hơn nhằm nâng cao tính ổn định, độ tin cậy và khả năng ứng dụng thực tế trong các mô hình bể cá thông minh hoặc hệ thống nuôi trồng thủy sản tự động. Để có thể hoàn thiện cũng như thương mại hóa được sản phẩm, cần phải nghiêm túc đặt ra những mục tiêu, hướng phát triển sau này cho hệ thống:

- p) Tăng cường bảo mật: Mã hóa gói tin trước khi gửi qua MQTT để đảm bảo an toàn dữ liệu trong quá trình truyền.
- q) Bổ sung cảm biến: Thêm cảm biến pH, nhiệt độ, oxy hòa tan (DO), và mực nước để đánh giá toàn diện chất lượng nước.
- r) Tự động kích hoạt bơm khi nước đục vượt ngưỡng.
- s) Lọc định kỳ hoặc bơm xả thay nước theo thời gian
- t) Tự động điều chỉnh lượng thức ăn dựa vào độ đục hoặc lịch định sẵn.
- u) Giảm thức ăn dư, giữ nước trong sạch.
- v) Xây dựng ứng dụng hoặc web hiển thị dữ liệu thời gian thực.
- w) Phát hiện sớm các điều kiện bất thường (qua phân tích dữ liệu cảm biến dài hạn)

KẾ HOẠCH PHÂN CÔNG NHIỆM VỤ VIẾT BÁO CÁO

MÔN TỬ KIẾN TRÚC VÀ GIAO THỨC IOT

HỌC KỲ 1 NĂM HỌC 2025–2026

3. Mã lớp môn học: ITAP338264_01

4. Tên đề tài: Thiết kế mô hình giám sát môi trường và cung cấp thức ăn tự động trong hồ cá tại nhà

5. Bảng phân công nhiệm vụ từ 06/09/2025 – 10/10/2025

STT	Nội dung hoàn thành	Thời gian (số tuần)	Người thực hiện
1	Tìm hiểu yêu cầu, khảo sát tài liệu về cảm biến (nhiệt độ, độ đục), cơ cấu cho ăn MQTT/Dashboard	1	Ngọc Dung, Hữu Dương, Duy Bình
2	Thiết kế kiến trúc hệ thống; lập BOM; sơ đồ nguyên lý, phương án nguồn & chống nhiễu; bố trí chân UART/I ² C/PWM	1	Duy Bình, Hữu Dương
3	Phát triển firmware STM32: ADC (độ đục), 1-Wire/NTC (nhiệt độ), PWM/servo feeder, nút/LED, lịch cho ăn cục bộ, khung UART (JSON/dòng)	2	Duy Bình, Ngọc Dung
4	Phát triển ESP8266/IoT: Wi-Fi, MQTT, bridge UART↔MQTT, hàng đợi khi mất mạng TLS	2	Hữu Dương
5	Dashboard: giao diện hiển thị nhiệt độ/độ đục/mức thức ăn, nút cho ăn, cảnh báo hết thức ăn	2	Ngọc Dung, Hữu Dương
6	Cơ khí: hoàn thiện bộ cho ăn, lắp vỏ chống ẩm, bố trí nguồn và đi dây	1	Duy Bình
7	Hiệu chuẩn và kiểm thử: đường cong ADC→NTU (≥5 điểm), kiểm tra nhiệt độ (≥3 điểm)	1	Ngọc Dung, Hữu Dương, Duy Bình
8	Viết, chỉnh sửa bài báo cáo Hoàn thiện mô hình phần cứng và code.	1	Ngọc Dung, Hữu Dương, Duy Bình

TÀI LIỆU THAM KHẢO

- [1] N. Hiền, "Quan, Thuế & Hải - Chuyên trang của tạp chí kinh tế - tài chính," 12 1 2024. [Online]. Available: <https://thuehaiquan.tapchikinhtetaichinh.vn//tphcm-xuat-khau-12-trieu-con-ca-can-h-mang-ve-125-trieu-usd-182372.html>. [Accessed 12 9 2025].
- [2] N. M. Đ. Diệp Thị Quế Ngân, "Hiện trạng nuôi cá cảnh giải trí của người dân thành phố Hồ Chí Minh," Bộ môn Quản Lý và Phát triển Nghề Cá, Đại Học Nông Lâm TP. Hồ Chí Minh, 2011.
- [3] R. Burhan, R. A. Rehman, B. Khan, and B. Kim, "IoT Elements, Layered Architectures, and Security Issues: A Comprehensive Survey," *Sensors*, vol. 18, no. 9, p. 2796, Aug. 2018. [Online]. Available: <https://doi.org/10.3390/s18092796>
- [3] "ISO 7027-1:2016 - Water quality — Determination of turbidity," International Organization for Standardization, 2016, <https://www.iso.org/standard/62801.html>
- [4] J. Trevathan, T. Atchison, L. Read, P. Hitchens, and W. Read, "Towards the Development of an Affordable and Practical Turbidity Sensor," *Sensors*, vol. 20, no. 7, p. 1993, 2020, <https://doi.org/10.3390/s20071993>
- [5] J. Droujko and P. Molnar, "Open-source, low-cost, in-situ turbidity sensor for river network monitoring," *Scientific Reports*, vol. 12, no. 10341, 2022, <https://doi.org/10.1038/s41598-022-14228-4>
- [6] R. Isoyama, M. Taie, T. Kageyama, M. Miura, A. Maeda, A. Mori, and S.-S. Lee, "A Feasibility Study on the Simultaneous Sensing of Turbidity and Chlorophyll a Concentration Using a Simple Optical Measurement Method," *Micromachines*, vol. 8, no. 4, p. 112, 2017, <https://doi.org/10.3390/mi8040112>
- [7] L. C. Sperandio, M. S. Colombo, C. M. G. Andrade, and C. B. B. Costa, "Development of a Low-Cost Portable Turbidimeter for Processes," arXiv preprint arXiv:2106.09491, 2021, <https://arxiv.org/abs/2106.09491>
- [8] "How to measure the turbidity in water using Arduino," EngineersGarage, Apr. 19, 2021. [Online]. Available: <https://www.engineersgarage.com/how-to-measure-the-turbidity-in-water-using-arduino/>
- [9] Google Firebase, "Cloud Firestore Overview," Firebase Documentation, 2025. [Online]. Available: <https://firebase.google.com/docs/firestore>